

# Lecture 4 - Designing a Professional I2C Driver Architecture

Bare-Metal STM32 Driver Development Course  
Target: STM32L452RE (Cortex-M4) — NUCLEO-L452RE

## Lecture 4 — Designing a Professional I2C Driver Architecture

**Course:** Bare-Metal STM32 Driver Development **Target MCU:** STM32L452RE — ARM Cortex-M4 **Board:** NUCLEO-L452RE **Scope of this lecture:** Architecture and API design **only** — no register-level implementation yet **Reference:** STM32L4 Series Reference Manual (RM0394) — used for hardware-behavior context, never copied verbatim

***A note before we begin:** Every professional firmware engineer who has shipped a peripheral driver into a real product will tell you the same thing — the code is the easy part. The hard part, the part that determines whether the driver survives five years of feature additions, bug fixes, and ports to new chips, is the **architecture decided before the first register write is ever typed**. This lecture is entirely dedicated to that decision-making process for our I2C driver. By the time Lecture 5 begins writing `I2C1->CR2`, every function name, every struct field, and every file will already have been justified here.*

### Learning Objectives

By the end of this lecture you will understand:

| #  | Objective                                                                                                  |
|----|------------------------------------------------------------------------------------------------------------|
| 1  | How firmware APIs are designed, from first principles, before any code exists                              |
| 2  | How to create reusable peripheral drivers that survive project growth                                      |
| 3  | How to design maintainable APIs with long lifespans                                                        |
| 4  | How to organize driver modules into a professional folder structure                                        |
| 5  | How to separate public and private interfaces cleanly                                                      |
| 6  | How professional SDKs (STM32Cube, Zephyr, Linux) are structured, and why                                   |
| 7  | Why configuration structures exist, and what belongs in one                                                |
| 8  | Why handles exist, and how they enable multi-instance drivers                                              |
| 9  | Why callbacks exist, and how they decouple drivers from applications                                       |
| 10 | Why state machines are necessary for correctness, not just style                                           |
| 11 | How applications interact with drivers, precisely, function by function                                    |
| 12 | How drivers interact with hardware, precisely, register by register (conceptually — no real registers yet) |

### Section 1 — What Is an API?

#### 1.1 Definition, From First Principles

An **API (Application Programming Interface)** is a **contract**: a documented, stable set of functions, data types, and behaviors that one piece of software promises to provide to another piece of software, without exposing *how* that promise is fulfilled internally. Strip away the acronym and the definition becomes very simple: **an API is a promise about behavior, not a description of implementation**.

***Real-World Analogy — The Car Pedal:** When you press a car’s accelerator pedal, you are using an API. The pedal promises “the car will accelerate proportionally to how far I am pressed.” You do not need to know whether the car is a V8 combustion engine, a turbocharged inline-4, or an electric motor. The mechanism underneath can change completely between car models — the pedal contract stays the same. This is exactly what `I2C_MasterTransmit()` must be to an application: a pedal, not an engine schematic.*

## 1.2 Why APIs Exist

APIs exist to solve a problem that appears the moment more than one person, or more than one module, must cooperate on a system: **without a contract, cooperation requires constant, exhausting communication about internal details**. An API removes that burden by letting each side trust a fixed interface instead of trusting the other side’s implementation choices.

In embedded systems specifically, APIs exist to:

1. **Freeze a stable boundary** between fast-moving application logic and slow-moving (but hardware-fragile) driver internals.
2. **Allow parallel development** — the application team can write code against an agreed-upon API before the driver is even finished, using stub implementations.
3. **Allow safe internal change** — the driver author can completely rewrite the internal register sequence (e.g., switch from polling to DMA) without breaking a single line of application code, as long as the API signature and behavior are preserved.
4. **Enable testing** — a well-defined API can be mocked, allowing application logic to be unit tested on a PC without real hardware.

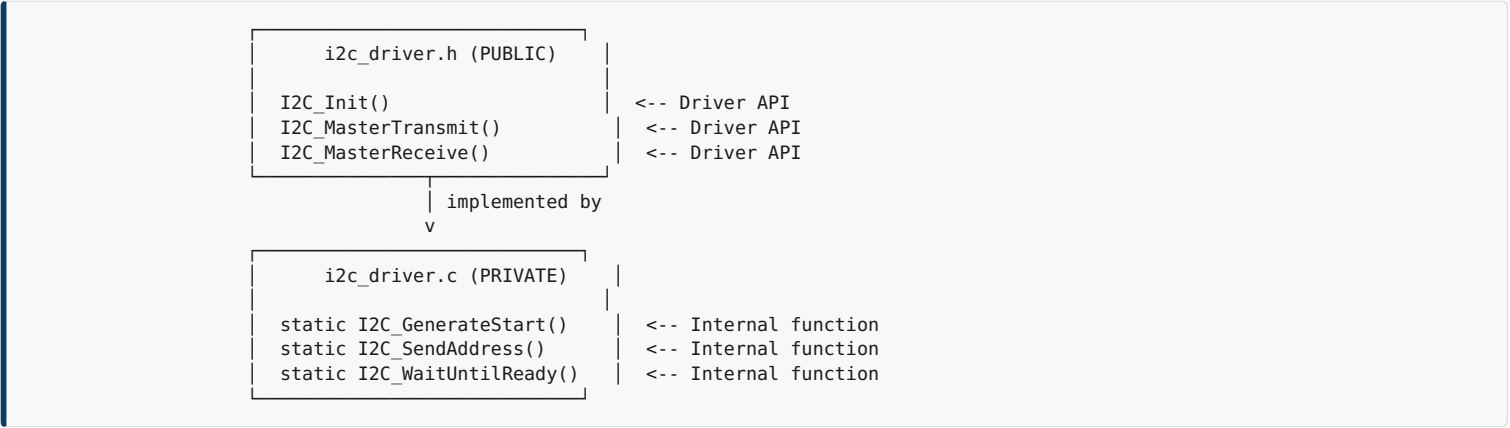
## 1.3 Hardware API vs Software API

| Aspect                 | Hardware API                                                                           | Software API                                                                        |
|------------------------|----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| What it exposes        | A physical peripheral’s behavior (e.g., “generate an I2C START condition”)             | A software module’s behavior (e.g., “sort this array”)                              |
| Underlying mechanism   | Registers, physical circuits, electrical signaling                                     | Data structures, algorithms, memory                                                 |
| Failure modes          | Bus timing violations, electrical noise, silicon errata                                | Logic bugs, invalid state, memory corruption                                        |
| Who defines it         | Silicon vendor (register map) <b>and</b> the driver author (the API you actually call) | The software author                                                                 |
| Example in this course | <code>I2C1-&gt;CR2</code> (the <i>hardware’s</i> API, defined by ST)                   | <code>I2C_MasterTransmit()</code> ( <i>our</i> API, built on top of the hardware’s) |

This distinction matters because **our driver’s public API is a software API that wraps a hardware API**. The application never sees the hardware API at all — that is the entire point of building a driver.

## 1.4 Driver API vs Internal Function

A **driver API** is the small set of functions the *rest of the firmware* is permitted to call. An **internal function** is a helper that exists purely to support the driver’s own implementation and must never be called from outside the driver module.



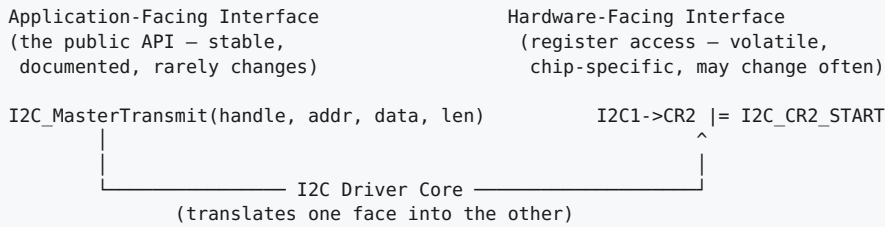
## 1.5 Public API vs Private API

- **Public API:** declared in the `.h` file, intended for consumption by application code, middleware, or other drivers. It is documented, versioned conceptually, and treated as a promise you should not casually break.
- **Private API:** declared `static` inside the `.c` file, invisible to the linker outside that translation unit, free to change at any time without affecting any other module.

**Why this distinction is not optional:** if every helper function were public, the driver would have dozens of entry points that application code *could* call in the wrong order, at the wrong time, or with the wrong assumptions — turning a controlled state machine into an uncontrollable one. Restricting visibility is a **safety mechanism**, not a stylistic preference.

## 1.6 Application-Facing Interface vs Hardware-Facing Interface

Every driver has exactly two “faces”:



The entire *purpose* of the driver’s internal code is to translate between these two faces. This is such an important idea that Section 2 builds an entire layered diagram around it.

### 1.7 API Contracts

An **API contract** is the complete, precise specification of a function’s behavior — not just its signature. A contract answers:

- What are the **preconditions**? (e.g., “the handle must have been successfully initialized”)
- What are the **postconditions**? (e.g., “on success, the data has been physically transmitted on the bus”)
- What happens on **invalid input**? (e.g., “returns `I2C_ERROR_INVALID_PARAM` if `data == NULL`”)
- What is **NOT guaranteed**? (e.g., “this function blocks the calling thread until completion or timeout — it is not interrupt-driven”)

A function signature alone, e.g. `I2C_Status_t I2C_MasterTransmit(I2C_Handle_t *h, uint16_t addr, uint8_t *data, uint16_t len);`, is **not** the contract — it is only the contract’s syntax. The full contract must be documented, and the implementation must honor it exactly, every time, or the API is worthless — callers stop trusting it and start reading the source code to “be sure,” which defeats the entire purpose of having an API.

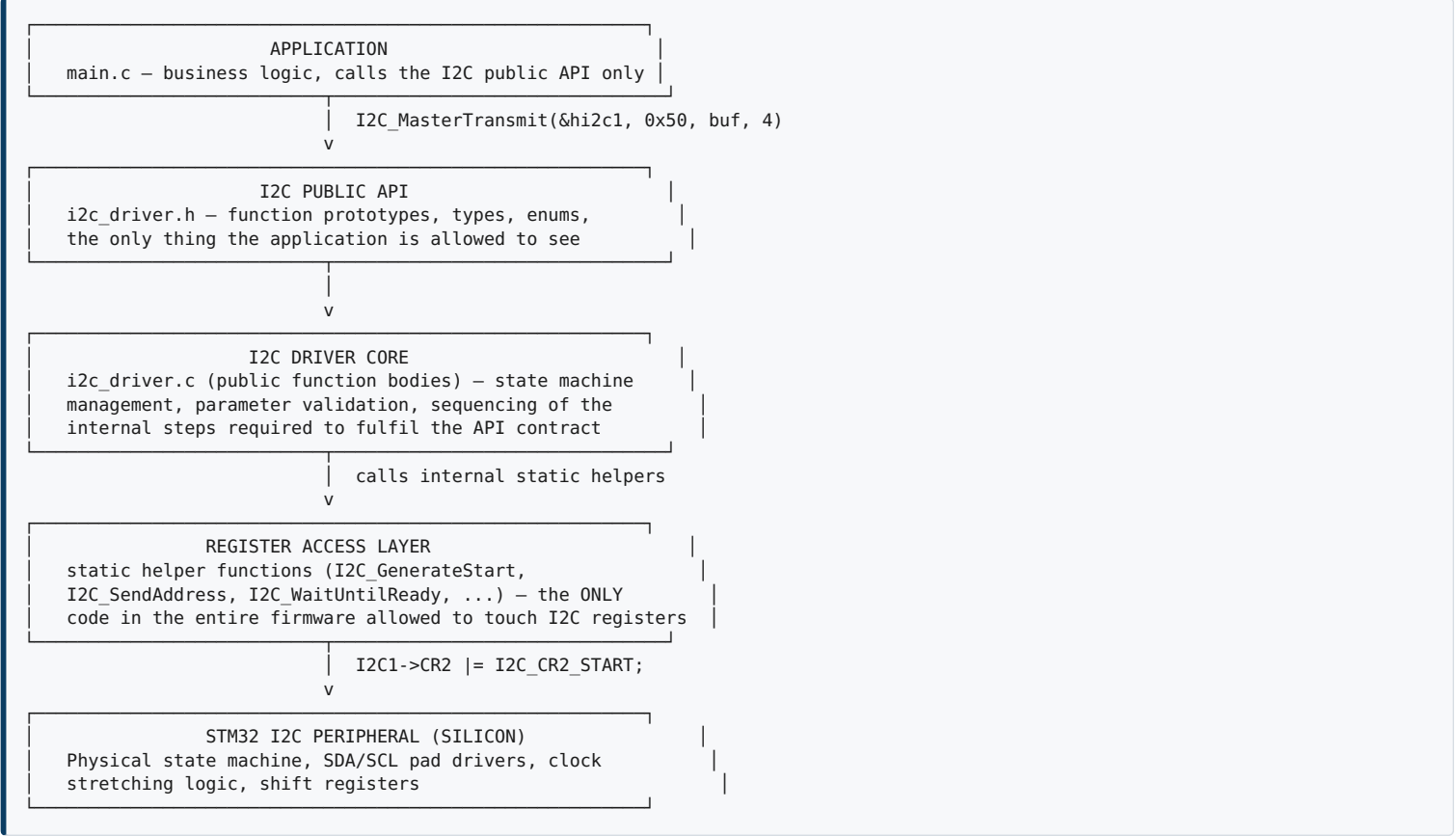
### 1.8 Encapsulation and Information Hiding

**Encapsulation** bundles data and the operations on that data into one unit. **Information hiding** (a closely related but distinct idea, coined by David Parnas in 1972) goes further: it says a module should hide **design decisions that are likely to change**, so that a change to one module does not ripple into every module that depends on it.

***Parnas’s Insight, Applied to I2C:** The exact algorithm used to compute `TIMINGR` register values for a given *SCL* frequency is exactly the kind of design decision Parnas would say to hide. If application code had to compute this itself, every future STM32 family with a different clock-tree architecture would break every application. By hiding this decision inside `I2C_Init()`, only the driver’s internals need to change when we port to a new chip.*

## Section 2 — Driver Architecture Before Coding

### 2.1 The Layered Model



## 2.2 Explaining Every Layer

- Layer 1 — Application.** Owns *business logic only*: “I need to write 4 bytes to device 0x50.” It has zero knowledge of I2C protocol timing, register names, or bit positions. If this layer needs to change (say, a new sensor is added), no other layer is affected.
- Layer 2 — I2C Public API.** This is a **pure interface layer** — it contains *declarations*, not logic. It exists as a physically separate concept (in `i2c_driver.h`) from Layer 3 so that the *promise* (contract) and the *fulfilment* (implementation) are visibly, architecturally distinct. Anyone can read this one file and understand everything the driver offers, without wading through implementation code.
- Layer 3 — I2C Driver Core.** This is where public function *bodies* live. This layer’s job is to: validate parameters, check the current state (Section 7 and Section 9 detail this), and orchestrate a *sequence* of lower-level steps to fulfil the contract. It does not decide *which exact bits* to set — that decision is delegated downward.
- Layer 4 — Register Access Layer.** This is the **only** layer in the entire firmware permitted to write `I2Cx->CRx`, `I2Cx->ISR`, etc. It is composed of small, single-purpose **static** functions. Concentrating all register access here means that if ST’s silicon has an erratum requiring a specific workaround, there is exactly **one place** in the whole codebase to fix it.
- Layer 5 — Hardware.** The physical I2C peripheral silicon — beyond firmware’s control, only observable and steerable through its registers.

## 2.3 Why Each Layer Exists — Summary Table

| Layer                 | Exists To...                                                              | If This Layer Didn’t Exist...                                                                                         |
|-----------------------|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Application           | Express <i>what</i> the product needs to do                               | Business logic and hardware detail would be inseparable — unmaintainable                                              |
| Public API            | Define a stable, minimal, documented contract                             | Every internal change would risk breaking every caller                                                                |
| Driver Core           | Orchestrate state, validation, and sequencing                             | Every caller would need to re-implement the same sequencing logic (and inevitably get it wrong differently each time) |
| Register Access Layer | Concentrate all hardware-specific bit manipulation in one auditable place | Errata fixes and register-level bugs would be scattered and duplicated across the codebase                            |
| Hardware              | Physically perform the communication                                      | N/A — this is the reason the whole stack exists                                                                       |

## 2.4 Advantages of This Layering

- Single Responsibility per layer** — each layer has exactly one reason to change.
- Testability** — the Driver Core’s state-machine logic can, in principle, be tested with a mocked Register Access Layer.
- Errata containment** — a chip revision bug affects only Layer 4.
- Team scalability** — one engineer can own Layer 4 (register-level correctness) while another owns Layer 3 (state machine and API correctness), with the boundary between them being the actual interface, not a verbal agreement.

5. **Auditable safety** — for safety-critical review, an auditor only needs to scrutinize Layer 4 for direct hardware correctness; Layers 1–3 can be reasoned about purely as software logic.

## Section 3 — Professional Driver Folder Structure

### 3.1 The Structure

```
Project_Root/
├── Drivers/
│   ├── Inc/
│   │   ├── stm32l452xx.h          <-- CMSIS device header (register defs)
│   │   ├── stm32l452xx_gpio.h    <-- GPIO driver public API
│   │   └── stm32l452xx_i2c.h     <-- I2C driver public API
│   └── Src/
│       ├── stm32l452xx_gpio.c    <-- GPIO driver implementation
│       └── stm32l452xx_i2c.c     <-- I2C driver implementation
├── Core/
│   ├── Inc/
│   │   └── system_config.h       <-- clock tree / system-wide config
│   └── Src/
│       ├── system_stm32l4xx.c    <-- CMSIS SystemInit(), clock setup
│       └── startup_stm32l452xx.s <-- vector table, Reset_Handler
├── Application/
│   ├── Inc/
│   │   └── app.h
│   └── Src/
│       └── app.c                 <-- application-level logic, sensor tasks
├── main.c                       <-- thin entry point
└── STM32L452RE_FLASH.ld         <-- linker script
```

### 3.2 Why This Organization Is Used

- **Drivers/Inc** vs **Drivers/Src** **split** mirrors the public/private distinction from Section 1.5 at the *filesystem* level, not just the language level — anyone browsing the repository immediately understands which files are “the contract” and which are “the implementation.”
- **One driver = one header + one source file pair.** This directly implements high cohesion and modularity (concepts introduced in Lecture 1) at the project-organization level.
- **Core/ is separated from Drivers/** because startup code and system clock configuration are *not* peripheral drivers — they are one-time, whole-system setup, and mixing them into the drivers folder would blur responsibility.
- **Application/ is separated from everything else** so that, in principle, the entire **Application/** folder could be deleted and replaced with a *different* application (a different product built on the same hardware and drivers) without touching **Drivers/** or **Core/** at all. This is the acid test of a properly layered project.

### 3.3 Dependency Flow

```
Application/  —depends on—>  Drivers/Inc/
                             |
                             | (Drivers/Src implements Drivers/Inc)
                             v
                             Drivers/Inc/  —depends on—>  Core/Inc/ (CMSIS device header)
                                                         |
                                                         v
                                                         CMSIS Core (ARM headers)
```

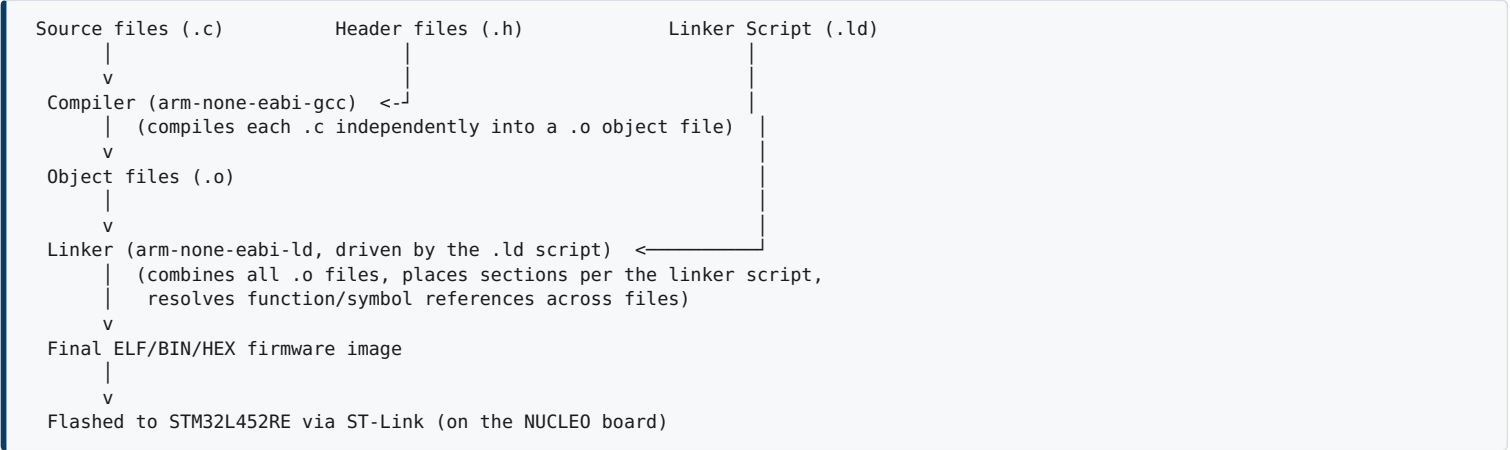
**Rule:** dependencies only ever point *downward* in this diagram. **Drivers/** must never **#include** anything from **Application/**. This is the file-system-level enforcement of the “no upward dependency” rule from Lecture 1’s Layer Separation principle.

### 3.4 Include Hierarchy

```
main.c
#include "app.h"
app.h
#include "stm32l452xx_i2c.h"
stm32l452xx_i2c.h
#include "stm32l452xx.h"    <-- CMSIS register definitions
stm32l452xx.h
#include "core_cm4.h"      <-- CMSIS core (ARM-provided)
```

Each header includes only what it directly needs (the principle of **minimal, explicit dependencies**), and header guards (**#ifndef** **STM32L452XX\_I2C\_H** / **#define** / **#endif**) prevent double-inclusion errors as this tree gets included from multiple places.

### 3.5 Build Process Overview



**Why this matters architecturally:** because compilation happens **per file**, any function not declared `static` is visible to the linker from *every other* object file in the project. This is the actual mechanical reason `static` internal functions (Section 6) genuinely cannot be called from outside their file — it is enforced by the compiler/linker, not just convention.

## Section 4 — API Design Philosophy

### 4.1 The Ten Pillars

| Principle                     | What It Means                                                                                                 | I2C Driver Example                                                                                                      |
|-------------------------------|---------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Simplicity</b>             | The API should be as small and easy to understand as the problem allows — no more                             | <code>I2C_MasterTransmit(handle, addr, data, len)</code> — four obviously-necessary parameters, nothing else            |
| <b>Readability</b>            | Function and parameter names should make the call site self-explanatory without needing to open documentation | <code>I2C_IsDeviceReady()</code> reads clearly; <code>I2C_Chk(h,1)</code> does not                                      |
| <b>Scalability</b>            | The API should accommodate growth (more instances, more modes) without redesign                               | Handle-based design (Section 7) scales to I2C1/I2C2/I2C3 for free                                                       |
| <b>Maintainability</b>        | A different engineer should be able to safely modify the implementation later                                 | Clear separation of Driver Core vs Register Access Layer                                                                |
| <b>Reusability</b>            | The same driver should work, unmodified, across multiple projects                                             | No project-specific assumptions baked into the driver files                                                             |
| <b>Backward compatibility</b> | Once shipped, an API’s existing behavior should not break for existing callers                                | Adding a new optional config field should never change what old code compiles to                                        |
| <b>Extensibility</b>          | New functionality should be addable without breaking the existing contract                                    | <code>I2C_RegisterCallback()</code> allows adding new event types later without new function signatures                 |
| <b>Error reporting</b>        | Every fallible operation must communicate failure, clearly and consistently                                   | Every API returns <code>I2C_Status_t</code> , never <code>void</code> , when failure is possible                        |
| <b>Defensive programming</b>  | Never trust input; validate before acting on it                                                               | <code>I2C_MasterTransmit()</code> checks <code>data != NULL</code> and <code>len &gt; 0</code> before touching hardware |
| <b>Consistent naming</b>      | A predictable naming scheme lets engineers <i>guess</i> correctly and be right                                | Every function: <code>I2C_&lt;Verb&gt;&lt;Noun&gt;()</code> , handle always first parameter                             |

### 4.2 Good vs Bad API Design — A Direct Comparison

| BAD API DESIGN                                                                                                                                                                                                                                                                                                                                             | GOOD API DESIGN                                                                                                                                                                                                                                                                                                                                                 |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>void i2c_go(int a, char* b, int c, int d);</pre>                                                                                                                                                                                                                                                                                                      | <pre>I2C_Status_t I2C_MasterTransmit(     I2C_Handle_t *handle,     uint16_t devAddress,     uint8_t *pData,     uint16_t size);</pre>                                                                                                                                                                                                                          |
| <b>Problems:</b> <ul style="list-style-type: none"> <li>- Unclear function purpose ("go" where?)</li> <li>- Unnamed, untyped parameters (a,b,c,d)</li> <li>- No return value – caller cannot detect failure</li> <li>- No handle – cannot support multiple I2C instances</li> <li>- "int" used for a device address (7-bit or 10-bit ambiguity)</li> </ul> | <b>Why it's better:</b> <ul style="list-style-type: none"> <li>- Verb+Noun name states exact behavior</li> <li>- Every parameter is named and typed</li> <li>- Returns status; caller can detect and handle failure</li> <li>- Handle enables I2C1/I2C2/I2C3 simultaneously</li> <li>- uint16_t clearly sized, matches 10-bit addressing possibility</li> </ul> |

4.3 A Second Example: Error Reporting Done Badly vs Well

| BAD: silent failure                                                                                                    | GOOD: explicit, actionable failure                                                                                                                                                                                                                             |
|------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>void I2C_MasterTransmit(...) {     if (busy) return; // caller never     // ... just hangs on failure ... }</pre> | <pre>I2C_Status_t I2C_MasterTransmit(...) {     if (handle-&gt;State != I2C_STATE_READY)         return I2C_ERROR_BUSY;     if (pData == NULL    size == 0)         return I2C_ERROR_INVALID_PARAM;     // ... perform transfer ...     return I2C_OK; }</pre> |

The “good” version costs a few extra lines. The “bad” version costs an entire debugging session, months later, in the field, when a caller cannot tell *why* their sensor reading silently never arrived.

Section 5 — Designing the Public API

This section is the architectural centerpiece of the lecture. For every function we **only design the contract** — no implementation.

5.1 I2C\_Init()

| Aspect                          | Specification                                                                                                                                                                                                                                                               |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Signature                       | <code>I2C_Status_t I2C_Init(I2C_Handle_t *handle);</code>                                                                                                                                                                                                                   |
| Purpose                         | Bring the I2C peripheral referenced by <code>handle-&gt;Instance</code> from an unconfigured/reset state to a fully configured, ready-to-operate state, using the parameters already stored in <code>handle-&gt;Config</code> .                                             |
| Parameters                      | <code>handle</code> — pointer to a caller-owned <code>I2C_Handle_t</code> , pre-filled with <code>Instance</code> and <code>Config</code> before this call.                                                                                                                 |
| Return type                     | <code>I2C_Status_t</code> — <code>I2C_OK</code> on success; <code>I2C_ERROR_INVALID_PARAM</code> if <code>handle</code> or <code>handle-&gt;Instance</code> is NULL; <code>I2C_ERROR_INVALID_CONFIG</code> if configuration values are out of supported range.              |
| Why it exists                   | Every peripheral needs an explicit, unambiguous “now it is ready” step — see the driver lifecycle from Lecture 1, Part 6.                                                                                                                                                   |
| When called                     | Exactly once per handle, after the handle’s <code>Config</code> fields have been set, before any transfer function is called.                                                                                                                                               |
| Internal operation (high level) | Enable the peripheral’s clock → configure associated GPIO pins to alternate function mode (delegating to the GPIO driver) → compute and write timing register values from the requested clock speed → set the handle’s <code>State</code> to <code>I2C_STATE_READY</code> . |
| Common mistakes                 | Calling <code>I2C_Init()</code> before setting <code>handle-&gt;Config</code> fields (uninitialized configuration); calling it more than once without an intervening <code>I2C_DeInit()</code> ; forgetting to check the return value.                                      |
| Example usage                   | <pre>c I2C_Handle_t hi2c1 = {.Instance = I2C1, .Config = {.ClockSpeed = 1000000}}; if (I2C_Init(&amp;hi2c1) != I2C_OK) { /* handle setup failure */ }</pre>                                                                                                                 |

5.2 I2C\_DeInit()

| Aspect             | Specification                                                                                                                                                   |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Signature          | <code>I2C_Status_t I2C_DeInit(I2C_Handle_t *handle);</code>                                                                                                     |
| Purpose            | Reverse everything <code>I2C_Init()</code> did: disable the peripheral, disable its clock, and return the handle to an uninitialized state.                     |
| Why it exists      | Symmetry (Lecture 1, Section 7.14) — every <code>Init</code> needs a <code>DeInit</code> so peripherals can be cleanly released, reconfigured, or powered down. |
| When called        | Before reconfiguring a peripheral for a different purpose, or before entering a low-power mode that requires the peripheral to be fully disabled.               |
| Internal operation | Disable the peripheral enable bit → disable the peripheral clock → set <code>handle-&gt;State = I2C_STATE_RESET</code> .                                        |
| Common mistakes    | Calling transfer functions after <code>DeInit</code> without re-calling <code>Init</code> first (should be caught by the state check).                          |

### 5.3 I2C\_Enable() / I2C\_Disable()

| Aspect                                     | Specification                                                                                                                                                                                                                                                                                             |
|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Signature                                  | <code>I2C_Status_t I2C_Enable(I2C_Handle_t *handle);</code> / <code>I2C_Status_t I2C_Disable(I2C_Handle_t *handle);</code>                                                                                                                                                                                |
| Purpose                                    | Toggle the peripheral enable bit <i>without</i> re-running the full initialization sequence — useful for temporarily pausing a bus without discarding configuration.                                                                                                                                      |
| Why they exist separately from Init/DeInit | <code>Init</code> / <code>DeInit</code> are “heavy” (full setup/teardown, including GPIO and clocks); <code>Enable</code> / <code>Disable</code> are “light” (just the peripheral enable bit) — collapsing these into one pair would force a full reconfiguration every time a temporary pause is needed. |
| Common mistakes                            | Confusing these with <code>Init</code> / <code>DeInit</code> and expecting GPIO/clock changes to also occur.                                                                                                                                                                                              |

### 5.4 I2C\_MasterTransmit()

| Aspect                          | Specification                                                                                                                                                                                                                                                                                     |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Signature                       | <code>I2C_Status_t I2C_MasterTransmit(I2C_Handle_t *handle, uint16_t devAddress, uint8_t *pData, uint16_t size, uint32_t timeout);</code>                                                                                                                                                         |
| Purpose                         | Send <code>size</code> bytes from <code>pData</code> to the slave device at <code>devAddress</code> , acting as I2C bus master.                                                                                                                                                                   |
| Why it exists                   | This is the single most common I2C operation — writing a command or data block to a sensor/EEPROM/peripheral device.                                                                                                                                                                              |
| When called                     | Any time after a successful <code>I2C_Init()</code> , while <code>handle-&gt;State == I2C_STATE_READY</code> .                                                                                                                                                                                    |
| Internal operation (high level) | Validate parameters and state → generate START condition → send address with write bit → wait for ACK → transmit each byte, waiting for the transmit-ready flag → generate STOP condition → restore state to ready.                                                                               |
| Common mistakes                 | Passing a stack buffer that goes out of scope before an asynchronous transfer completes (irrelevant for the blocking version designed here, but a classic mistake once interrupt/DMA modes are added in Section 10); ignoring the timeout parameter and assuming the call always returns quickly. |
| Example usage                   | <code>c uint8_t cmd[2] = {0x01, 0xFF}; if (I2C_MasterTransmit(&amp;hi2c1, 0x50, cmd, 2, 100) != I2C_OK) { /* retry or log */ }</code>                                                                                                                                                             |

### 5.5 I2C\_MasterReceive()

| Aspect             | Specification                                                                                                                                                                                     |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Signature          | <code>I2C_Status_t I2C_MasterReceive(I2C_Handle_t *handle, uint16_t devAddress, uint8_t *pData, uint16_t size, uint32_t timeout);</code>                                                          |
| Purpose            | Receive <code>size</code> bytes from the slave device at <code>devAddress</code> into <code>pData</code> , as bus master.                                                                         |
| Internal operation | Generate START → send address with read bit → wait for ACK → read each byte (correctly NACK-ing the final byte per the I2C protocol) → generate STOP.                                             |
| Common mistakes    | Forgetting that the <i>last</i> received byte must be NACKed rather than ACKed (an I2C protocol subtlety directly relevant to register sequencing in Lecture 5); passing <code>size == 0</code> . |

### 5.6 I2C\_MemWrite() / I2C\_MemRead()



| Aspect                                              | Specification                                                                                                                                                                                                                                                                                                                                                                                  |
|-----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Signature                                           | <code>I2C_Status_t I2C_MemWrite(I2C_Handle_t *handle, uint16_t devAddress, uint16_t memAddress, uint8_t memAddrSize, uint8_t *pData, uint16_t size, uint32_t timeout);</code> (and the mirrored <code>I2C_MemRead</code> )                                                                                                                                                                     |
| Purpose                                             | A very common I2C usage pattern is not “just send bytes” but “write/read at a specific internal register/memory address of the target device” (e.g., EEPROMs, sensor register maps). These functions exist because <i>combining</i> an internal address write with a data transfer is common enough, and error-prone enough to hand-roll every time, that it deserves its own first-class API. |
| Why not just use <code>MasterTransmit</code> twice? | Between writing the memory address and reading the data, an I2C <b>repeated START</b> (not a STOP followed by a new START) is required by many devices to maintain bus ownership. This subtlety is exactly the kind of protocol detail Section 1.8’s “hide design decisions likely to be gotten wrong” principle tells us to encapsulate.                                                      |
| <code>memAddrSize</code> parameter                  | Supports devices with 8-bit internal addresses (most EEPROMs) vs 16-bit internal addresses (larger EEPROMs) without needing two separate function names.                                                                                                                                                                                                                                       |

### 5.7 `I2C_ScanBus()`

| Aspect          | Specification                                                                                                                                                                                                                      |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Signature       | <code>uint8_t I2C_ScanBus(I2C_Handle_t *handle, uint8_t *foundAddresses, uint8_t maxCount);</code>                                                                                                                                 |
| Purpose         | Probe every valid 7-bit address on the bus and report which ones ACK, returning the count found. A diagnostic/development-time utility.                                                                                            |
| Why it exists   | Extremely useful during bring-up and debugging of new hardware — lets an engineer discover what is actually connected before writing device-specific code.                                                                         |
| Common mistakes | Leaving bus-scan code enabled in production firmware (wastes flash and time, and can confuse devices sensitive to unexpected addressing) — should typically be compiled out or gated behind a debug build flag (see Section 9.10). |

### 5.8 `I2C_IsDeviceReady()`

| Aspect                                      | Specification                                                                                                                                                                                                                                                |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Signature                                   | <code>I2C_Status_t I2C_IsDeviceReady(I2C_Handle_t *handle, uint16_t devAddress, uint32_t trials, uint32_t timeout);</code>                                                                                                                                   |
| Purpose                                     | Poll a specific device address repeatedly (up to <code>trials</code> times) until it ACKs, useful for devices that take time to power up or finish an internal operation (e.g., EEPROM write cycles).                                                        |
| Why it’s distinct from <code>ScanBus</code> | <code>ScanBus</code> answers “what’s out there” (development-time); <code>IsDeviceReady</code> answers “is <i>this specific</i> known device ready <i>right now</i> ” (production-time, e.g., waiting for an EEPROM write to complete before the next read). |

### 5.9 `I2C_ResetBus()`

| Aspect        | Specification                                                                                                                                                                                                                                   |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Signature     | <code>I2C_Status_t I2C_ResetBus(I2C_Handle_t *handle);</code>                                                                                                                                                                                   |
| Purpose       | Perform bus-recovery: forcibly clear a stuck bus condition (SDA held low by a confused slave) by manually toggling SCL as a GPIO, per the standard I2C bus-recovery procedure, then reinitializing the peripheral.                              |
| Why it exists | I2C, unlike SPI, allows any slave to hold the bus (via clock stretching or a stuck SDA), so a production-quality driver <b>must</b> have a documented recovery path — this was flagged as a required capability back in Lecture 1, Section 8.8. |
| When called   | Automatically, by the Driver Core, after detecting a fatal bus-level error (Section 9); may also be exposed for manual invocation by application-level watchdog logic.                                                                          |

### 5.10 `I2C_GetState()` / `I2C_GetError()` / `I2C_ClearError()`

| Aspect          | Specification                                                                                                                                                                                                                             |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Signatures      | <code>I2C_State_t I2C_GetState(I2C_Handle_t *handle); / I2C_Error_t I2C_GetError(I2C_Handle_t *handle); / void I2C_ClearError(I2C_Handle_t *handle);</code>                                                                               |
| Purpose         | Give application code <b>read-only visibility</b> into the driver’s internal state machine and last error, without exposing the handle’s internals directly for modification.                                                             |
| Why they exist  | A return code tells you about <i>one call</i> . <code>GetState / GetError</code> let you inspect the <i>ongoing</i> condition of the peripheral at any time — essential for building robust application-level retry and diagnostic logic. |
| Common mistakes | Application code directly reading <code>handle-&gt;State</code> instead of calling <code>I2C_GetState()</code> — this bypasses encapsulation and breaks the moment the handle’s internal layout changes.                                  |

5.11 I2C\_RegisterCallback()

| Aspect                                                    | Specification                                                                                                                                                                                                                                                                                                                                      |
|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Signature                                                 | <code>I2C_Status_t I2C_RegisterCallback(I2C_Handle_t *handle, I2C_CallbackEvent_t event, I2C_Callback_t callback);</code>                                                                                                                                                                                                                          |
| Purpose                                                   | Allow application code to supply a function pointer that the driver will invoke when a specific event occurs (transfer complete, error, etc.) — critical groundwork for interrupt/DMA-driven operation in later lectures.                                                                                                                          |
| Why it exists now, even before interrupts are implemented | Designing the callback <b>registration mechanism</b> as part of the architecture <i>today</i> means the blocking (polling) implementation we build first in Lecture 5 will not need an incompatible API redesign when Lecture 7 or 8 introduces interrupt-driven operation — this is <b>Section 10’s</b> scalability principle applied concretely. |
| Common mistakes                                           | Registering a callback that itself calls back into <code>I2C_MasterTransmit()</code> on the <i>same</i> handle from <i>within</i> the callback context — a classic re-entrancy bug we will flag again in Section 9.                                                                                                                                |

Section 6 — Designing Internal Driver Functions

6.1 Why Internal Helper Functions Must Not Be Public

- Internal helper functions perform the *individual mechanical steps* that public API functions sequence together. If these were public:
- Application code could call `I2C_GenerateStart()` alone, leaving the bus in an inconsistent state (a START with no address, no data, no STOP).
  - The driver’s state machine (Section 7, Section 11) could be bypassed entirely, since these helpers don’t perform the state validation the public functions do.
  - Every internal refactor (e.g., changing how `I2C_WaitUntilReady()` polls) would risk breaking external callers who “discovered” and started depending on the internal function.

6.2 The Internal Function Set

```
static void    I2C_GenerateStart(I2C_Handle_t *handle);
static void    I2C_GenerateStop(I2C_Handle_t *handle);
static I2C_Status_t I2C_SendAddress(I2C_Handle_t *handle, uint16_t devAddress, uint8_t direction);
static I2C_Status_t I2C_SendByte(I2C_Handle_t *handle, uint8_t byte);
static uint8_t I2C_ReadByte(I2C_Handle_t *handle, uint8_t ackRequired);
static I2C_Status_t I2C_WaitUntilReady(I2C_Handle_t *handle, uint32_t flagMask, uint32_t timeout);
static void    I2C_ClearFlags(I2C_Handle_t *handle);
static void    I2C_HandleError(I2C_Handle_t *handle, I2C_Error_t error);
```

| Function                        | Single Responsibility                                                                                               |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <code>I2C_GenerateStart</code>  | Produce exactly one hardware action: assert the START condition                                                     |
| <code>I2C_GenerateStop</code>   | Produce exactly one hardware action: assert the STOP condition                                                      |
| <code>I2C_SendAddress</code>    | Frame and transmit the 7/10-bit address plus R/W bit, and confirm ACK                                               |
| <code>I2C_SendByte</code>       | Transmit exactly one data byte, waiting for the hardware to be ready                                                |
| <code>I2C_ReadByte</code>       | Receive exactly one data byte, correctly controlling ACK/NACK                                                       |
| <code>I2C_WaitUntilReady</code> | Poll a status flag with a bounded timeout — the <b>only</b> place polling logic is written                          |
| <code>I2C_ClearFlags</code>     | Clear any latched status flags that must be explicitly cleared before further operation                             |
| <code>I2C_HandleError</code>    | Centralize what happens when something goes wrong: update state, record the error code, optionally trigger recovery |

### 6.3 Function Call Hierarchy — I2C\_MasterTransmit Internals

```
I2C_MasterTransmit(handle, addr, data, len, timeout)    <-- PUBLIC (Driver Core)
├── validate parameters, validate handle->State
├── I2C_GenerateStart(handle)                            <-- static, Register Access Layer
│   └── I2C1->CR2 |= I2C_CR2_START;    (conceptual – real bits in Lecture 5)
├── I2C_WaitUntilReady(handle, SB_FLAG, timeout)        <-- static
├── I2C_SendAddress(handle, addr, WRITE)                <-- static
│   ├── I2C1->TXDR = framedAddress;
│   └── I2C_WaitUntilReady(handle, ADDR_FLAG, timeout)
├── for each byte in data:
│   ├── I2C_WaitUntilReady(handle, TXE_FLAG, timeout)
│   └── I2C_SendByte(handle, data[i])
├── I2C_WaitUntilReady(handle, BTF_FLAG, timeout)
├── I2C_GenerateStop(handle)
└── on any failure at any step -> I2C_HandleError(handle, errorCode); return errorCode;
```

This diagram is the concrete proof of Section 2’s layering: **the public function never touches a register directly** — it only calls named, single-purpose internal helpers, each of which touches exactly the registers its name promises.

### 6.4 Module Visibility — The C Mechanism Behind All of This

In C, `static` at file scope means “this symbol has internal linkage — it does not appear in the object file’s exported symbol table, and no other translation unit can link against it.” This is not a convention enforced by discipline alone — it is enforced by the compiler and linker. Attempting to call a `static` function from `main.c` produces a **compile error** (“implicit declaration”) or a **link error** (“undefined reference”), not just a code-review complaint.

```
i2c_driver.c compiled to i2c_driver.o:

Exported symbols (visible to linker, callable from other files):
  I2C_Init
  I2C_DeInit
  I2C_MasterTransmit
  I2C_MasterReceive
  ...

Internal symbols (NOT exported – invisible outside this .o file):
  I2C_GenerateStart    <-- static
  I2C_SendAddress      <-- static
  I2C_WaitUntilReady   <-- static
```

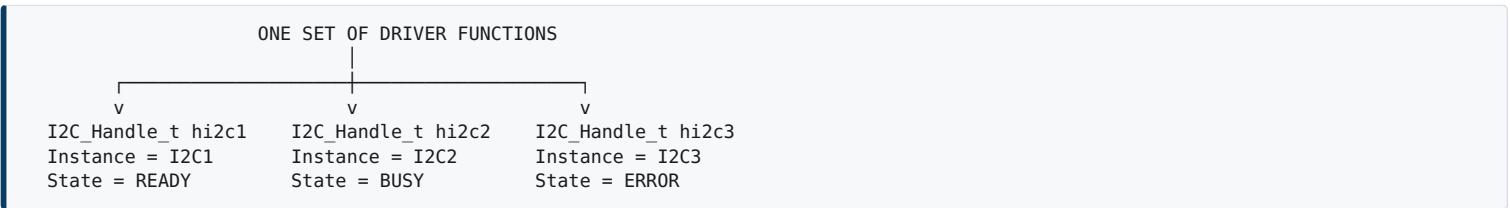
## Section 7 — Driver Configuration Philosophy

### 7.1 Why Configuration Objects Exist

A configuration object exists to answer one question cleanly: **“What does the caller want, before anything happens?”** Bundling every user-adjustable parameter into one struct (rather than a long parameter list, per Lecture 1 Section 7.8) means the API can grow new options over time without breaking every existing call site, provided sensible defaults are documented for any new field.

### 7.2 Why Handles Exist

A handle exists to answer a different question: **“Which specific peripheral instance, and what is its current condition, right now?”** Without a handle, a driver can only ever control one hardware instance (a common beginner-driver mistake), because there would be nowhere to store *which* `I2Cx` block a given function call refers to, or what state that instance is currently in.



The **same** `I2C_MasterTransmit()` function body operates correctly on any of these three handles, because all instance-specific and state-specific information is carried *in the handle argument*, not hardcoded into the function.

### 7.3 Why State Variables Exist

A state variable exists to prevent the driver from being asked to do something that is currently unsafe or undefined — e.g., starting a second transmission while the first has not completed. This is the enforcement mechanism for the state machine concept introduced in Lecture 1 (Section 7.11) and detailed further in Section 8 of this lecture.

### 7.4 Why Callback Pointers Exist

A callback pointer exists to let the driver **notify** the application of an event (completion, error) *without the driver needing to know anything about what the application wants to do in response*. This is inversion of control: the driver defines *when* it will call out, the application defines *what happens* when it does.

### 7.5 Why Timeout Values Should Be Configurable

Different use cases have wildly different acceptable latencies: a factory-floor sensor read might tolerate a 100ms timeout easily, while a hard real-time control loop might require a 2ms bound. Hardcoding a timeout inside the driver would force every user of the driver to accept one project’s assumptions. Making timeout a parameter (or a configuration field with a sensible default) keeps the driver reusable across very different products — directly serving the Reusability pillar from Section 4.1.

### 7.6 Compile-Time vs Run-Time Configuration

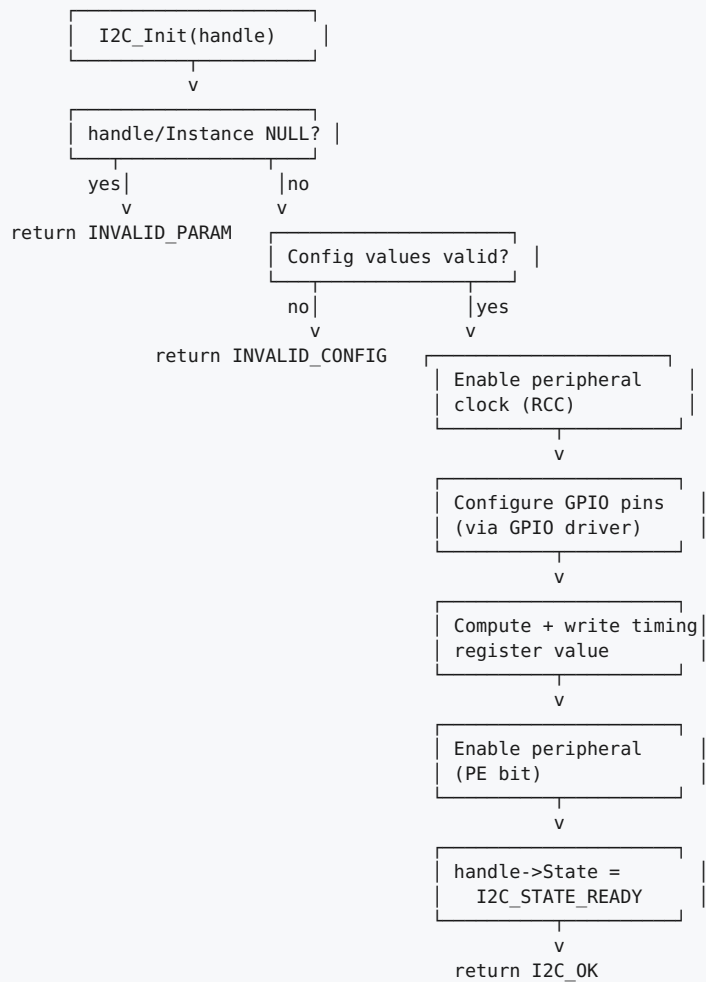
|             | Compile-Time Configuration                                                                                                              | Run-Time Configuration                                                                                                       |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Mechanism   | <code>#define</code> , conditional compilation ( <code>#ifdef</code> ), constant structs in flash                                       | Fields inside a struct set by code before <code>I2C_Init()</code>                                                            |
| Example     | <code>#define I2C_MAX_INSTANCES 3</code> — fixed at build time, cannot change without recompiling                                       | <code>handle-&gt;Config.ClockSpeed = 400000;</code> — chosen by the calling application at runtime                           |
| When to use | Resource limits, feature toggles that will never change on a given target, safety-critical constants that should not be runtime-mutable | Anything the application legitimately needs to decide based on its own logic (which sensor is attached, what speed it needs) |
| Trade-off   | Smaller/faster code, but requires recompilation to change; safer against accidental runtime corruption                                  | More flexible, but consumes RAM and must be validated at runtime                                                             |

### 7.7 Default Configurations

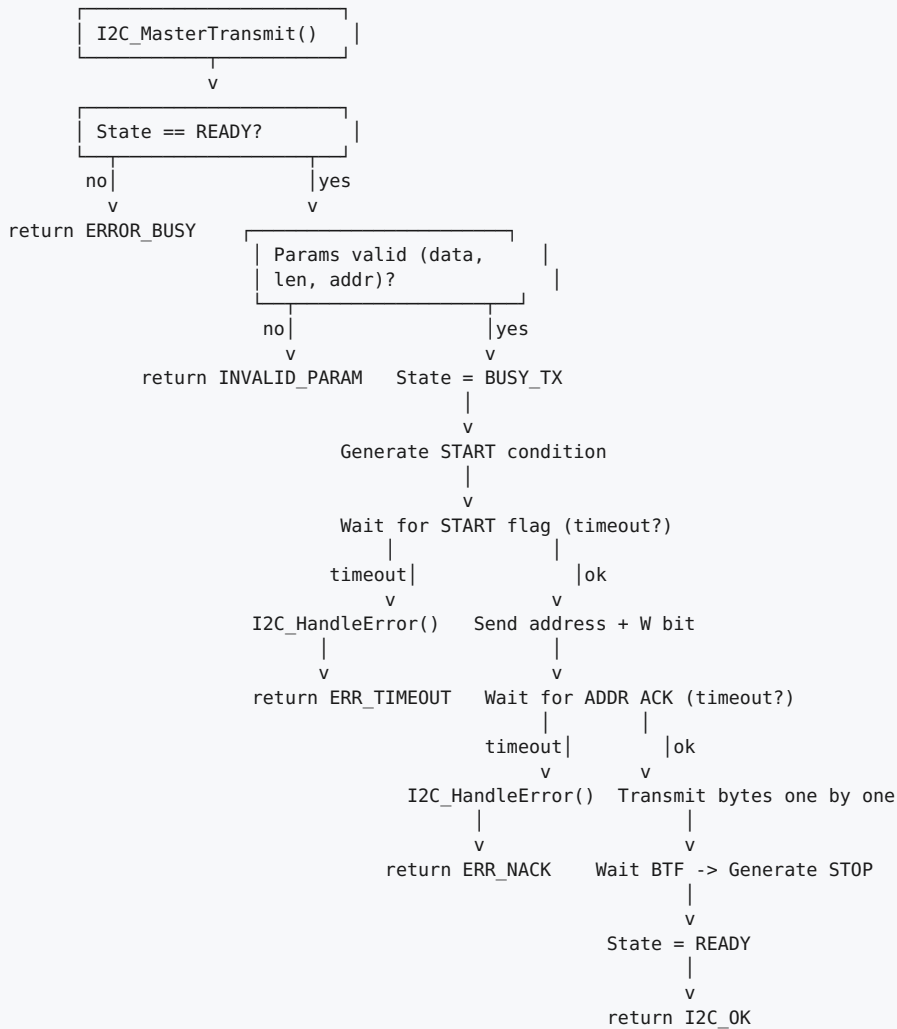
Professional drivers typically ship a documented **default configuration** (e.g., `I2C_GetDefaultConfig(&cfg)` or a `static const I2C_Config_t I2C_DEFAULT_CONFIG`), so a new user can get a working peripheral quickly ( `cfg = I2C_DEFAULT_CONFIG; cfg.ClockSpeed = 400000; I2C_Init(&handle, &cfg);` ) while still being able to override only the fields they care about. This reduces the “wall of required parameters” problem and lowers the barrier to correct first use — an underrated but very real API usability factor.

## Section 8 — Driver Execution Flow

### 8.1 Driver Initialization Flowchart



## 8.2 Master Transmit Flowchart



### 8.3 Master Receive Flowchart (Condensed)

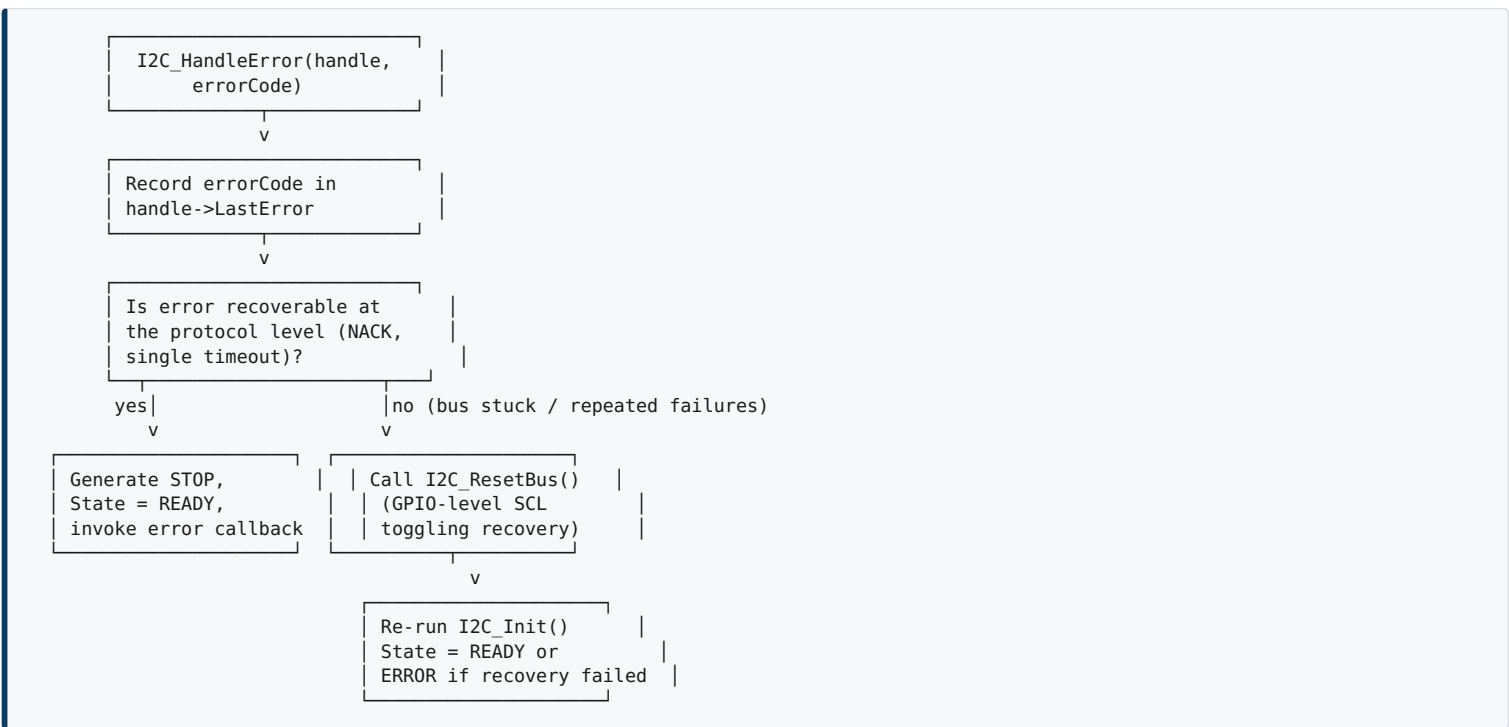
```
I2C_MasterReceive()  
├─ State==READY? —no—> return ERROR_BUSY  
├─ Params valid? —no—> return INVALID_PARAM  
├─ State = BUSY_RX  
├─ Generate START  
├─ Send address + R bit  
├─ Wait ADDR ACK  
├─ For each byte except the last:  
│   └─ Wait RXNE -> Read byte -> Send ACK  
├─ For the last byte:  
│   └─ Wait RXNE -> Send NACK -> Read byte  
├─ Generate STOP  
├─ State = READY  
└─ return I2C_OK
```

**Design Note:** The asymmetry in the last step (NACK before reading the final byte, not after) is a real I2C protocol requirement, not an arbitrary choice — flagging it here, architecturally, ensures Lecture 5’s register-level implementation gets this exactly right rather than discovering it as a bug during hardware bring-up.

### 8.4 Memory Write / Memory Read Flowchart (Condensed)

```
I2C_MemWrite(handle, devAddr, memAddr, memAddrSize, data, len, timeout)  
├─ State==READY? / Params valid? (as above)  
├─ Generate START  
├─ Send devAddr + W bit, wait ACK  
├─ Send memAddr bytes (1 or 2, per memAddrSize), wait ACK each  
├─ Transmit data bytes (identical inner loop to MasterTransmit)  
├─ Generate STOP  
└─ return I2C_OK  
  
I2C_MemRead(handle, devAddr, memAddr, memAddrSize, data, len, timeout)  
├─ Generate START  
├─ Send devAddr + W bit, wait ACK <-- write phase: select the address  
├─ Send memAddr bytes, wait ACK  
├─ Generate REPEATED START (no STOP first!) <-- protocol-critical detail  
├─ Send devAddr + R bit, wait ACK  
├─ Read data bytes (identical inner loop to MasterReceive, NACK on last)  
├─ Generate STOP  
└─ return I2C_OK
```

### 8.5 Error Recovery Flowchart



### 8.6 Driver Shutdown Flowchart

```
I2C_DeInit(handle)
|
├─ State == BUSY? —yes—> return ERROR_BUSY (do not tear down mid-transfer)
├─ Disable peripheral (PE bit = 0)
├─ Disable peripheral clock (RCC)
├─ (Optionally) release GPIO pins to analog/default state
├─ handle->State = I2C_STATE_RESET
└─ return I2C_OK
```

## 8.7 Full API Call Flow — `main()` to Hardware

```
main()
|
v
System_Init()                <-- clock tree, SysTick
|
v
I2C_Init(&hi2c1)             <-- APPLICATION calls PUBLIC API
|
v
[Driver Core] validates, sequences setup steps
|
v
[Register Access Layer] RCC->APB1ENR1 |= RCC_APB1ENR1_I2C1EN; (conceptual)
|                      I2C1->TIMINGR = computedValue;
|                      I2C1->CR1 |= I2C_CR1_PE;
|
v
[Hardware] I2C1 peripheral silicon transitions from reset to enabled state
|
v
main() continues:
I2C_MasterTransmit(&hi2c1, 0x50, buf, 4, 100)
|
v
[Driver Core -> Register Access Layer -> Hardware] (as traced in Section 6.3)
|
v
main() reads return value, decides next action, loops
```

# Section 9 — Error Handling Strategy

## 9.1 Return Codes vs Status Codes vs Error Codes — A Careful Distinction

These three terms are often used interchangeably by beginners, but a professional architecture treats them as three related, distinct concepts:

| Term        | Meaning                                                                                            | Example Type                                                                                                                                                                               |
|-------------|----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Return code | The immediate outcome of <i>this specific function call</i>                                        | <code>I2C_Status_t</code> returned by <code>I2C_MasterTransmit()</code>                                                                                                                    |
| Status      | The <i>ongoing condition</i> of the peripheral, queryable at any time, independent of any one call | <code>I2C_State_t</code> returned by <code>I2C_GetState()</code> ( <code>READY</code> , <code>BUSY_TX</code> , <code>BUSY_RX</code> , <code>ERROR</code> )                                 |
| Error code  | The <i>specific reason</i> the last failure occurred, persisted until explicitly cleared           | <code>I2C_Error_t</code> returned by <code>I2C_GetError()</code> ( <code>NONE</code> , <code>TIMEOUT</code> , <code>NACK</code> , <code>BUS_ERROR</code> , <code>ARBITRATION_LOST</code> ) |

A return code answers “did *this call* work?” A status answers “what is the peripheral doing *right now*?” An error code answers “*why* did the last failure happen, in detail?” All three are necessary; none can substitute for the others.

## 9.2 Timeout Handling

Every blocking wait in the driver (Section 6’s `I2C_WaitUntilReady`) must be bounded by a timeout, measured against a monotonic time source (e.g., `SysTick`-derived tick counter). **No wait loop in a professional driver is allowed to be unbounded** — an unbounded wait converts a single misbehaving slave device into a permanently hung system, which is unacceptable in any product that must remain responsive or safety-relevant.

## 9.3 Retry Mechanisms

Some failures (a single NACK during bus arbitration, a transient timeout) are appropriately retried a small, bounded number of times before being reported as a hard failure. Retry logic belongs at the **Driver Core** layer (not the Register Access Layer, which has no concept of “try again” — it only performs one mechanical step), and the retry count/backoff should be configurable rather than a hardcoded magic number.

## 9.4 Bus Recovery

Already introduced via `I2C_ResetBus()` (Section 5.9) and diagrammed in Section 8.5. Architecturally, bus recovery is the **last resort** in the error-handling hierarchy — attempted only after simpler recoverable errors (single NACK, single timeout) have been ruled out.

## 9.5 Invalid Parameters

Every public API function must validate its parameters as the **very first** action, before touching any hardware state, and return `I2C_ERROR_INVALID_PARAM` immediately on failure. This is non-negotiable defensive programming (Section 4.1) — a NULL pointer or zero-length transfer must never reach the Register Access Layer.

## 9.6 Busy Peripheral Handling

A public API function must check `handle->State` before acting. If the peripheral is already busy with a prior operation, the correct response is an immediate `I2C_ERROR_BUSY` return — **never** silently queueing, blocking indefinitely, or (worst of all) proceeding and corrupting the in-flight transfer.

## 9.7 Assertions

Assertions ( `I2C_ASSERT(condition)` ) are used to catch **programmer errors** — conditions that should be logically impossible if the driver itself is implemented correctly (e.g., an internal function receiving a NULL handle that the public function should have already rejected). Assertions differ from return-code error handling in an important way:

| Return Codes               |                                                         | Assertions                                                           |
|----------------------------|---------------------------------------------------------|----------------------------------------------------------------------|
| Catch                      | Expected runtime failures (bad input, hardware timeout) | Impossible-if-correct internal logic violations                      |
| Present in release builds? | Always                                                  | Often disabled/stripped in release builds for performance            |
| Typical response           | Return an error code, let the caller decide             | Halt execution (in debug builds) to catch the bug during development |

## 9.8 Debug Builds vs Release Builds

A professional driver typically supports a build-time flag (e.g., `I2C_DEBUG` ) that enables:

- Assertions (Section 9.7).
- Optional verbose internal logging (e.g., via a weakly-linked `I2C_Debug_Log()` hook).
- Development-time utilities like `I2C_ScanBus()` (Section 5.7), which may be excluded entirely from release builds to save flash space and avoid unintended bus traffic in production.

```
#ifdef I2C_DEBUG
#define I2C_ASSERT(cond) do { if (!(cond)) { I2C_AssertFailed(__FILE__, __LINE__); } } while(0)
#else
#define I2C_ASSERT(cond) ((void)0)
#endif
```

## 9.9 The Complete Error Enum (Conceptual Design)

```
typedef enum
{
    I2C_OK = 0,
    I2C_ERROR_INVALID_PARAM,
    I2C_ERROR_INVALID_CONFIG,
    I2C_ERROR_BUSY,
    I2C_ERROR_TIMEOUT,
    I2C_ERROR_NACK,
    I2C_ERROR_BUS_ERROR,
    I2C_ERROR_ARBITRATION_LOST,
    I2C_ERROR_NOT_INITIALIZED,
    I2C_ERROR_RECOVERY_FAILED
} I2C_Status_t;
```

Each value maps to a specific, documented cause — never a generic `I2C_ERROR_UNKNOWN` catch-all in a well-designed driver (a catch-all error code is itself a design smell, discussed further in Section 14).

# Section 10 — Driver Scalability

## 10.1 The Scalability Requirement

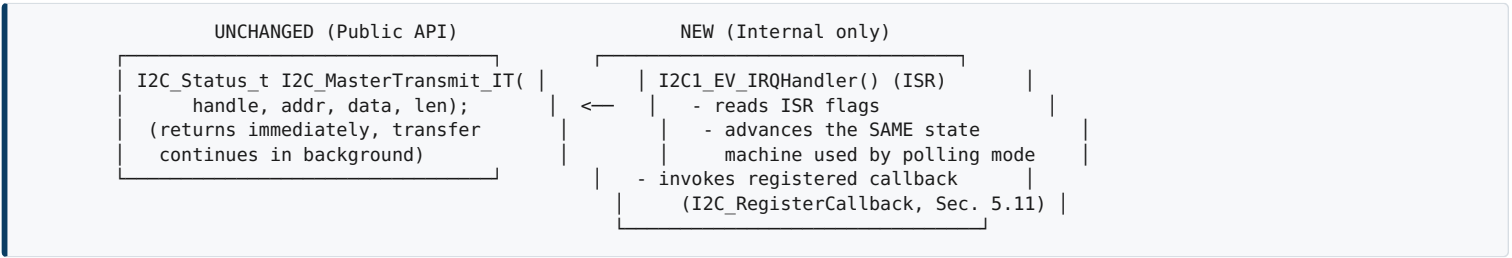


A driver designed only for “today’s” polling-mode, single-instance use case will require an **API-breaking rewrite** the moment interrupts, DMA, or a second I2C instance are needed — unless scalability is designed in from the start. This section shows *why* the architecture from Sections 5–9 already accommodates every extension below **without changing the public API surface**.

### 10.2 Polling Mode (Today’s Design)

The blocking implementations sketched in Section 8 ( `I2C_MasterTransmit` , etc.) are the polling-mode baseline. The caller’s function call does not return until the operation completes or times out.

### 10.3 Interrupt Mode — What Changes, What Doesn’t



Because Section 7 already designed the handle to carry `State` , buffer pointers, and remaining-length counters (needed for *any* multi-step operation, polling or not), and Section 5.11 already designed the callback registration mechanism, adding an interrupt-mode entry point is an **additive** change — a new function name ( `_IT` suffix, following STM32Cube’s own convention, discussed in Section 12), not a redesign of existing ones.

### 10.4 DMA Mode

Similarly, a future `I2C_MasterTransmit_DMA()` would reuse the same handle, the same state machine, and the same callback mechanism — only the Register Access Layer changes, to configure a DMA channel to feed the I2C data register instead of the CPU writing each byte in a polling loop.

### 10.5 RTOS Integration

Under an RTOS, the Driver Core would additionally guard `handle->State` transitions with a mutex (protecting against two tasks calling into the same handle concurrently — flagged as Interview Question 30 territory in Lecture 1), and a blocking call could internally use a semaphore given from the ISR instead of a polling loop — again, a Driver-Core-and-Register-Access-Layer-only change; the public API signature for the blocking call does not need to change at all.

### 10.6 Multi-Instance Support (I2C1/I2C2/I2C3)

Already fully solved by the handle design (Section 7.2) — three independent `I2C_Handle_t` instances, each with its own `Instance` pointer and `State` , work correctly with zero additional code, because no function relies on a hardcoded `I2C1` reference anywhere above the Register Access Layer.

### 10.7 Future STM32 Families

If a future STM32 family (e.g., STM32H5) has a different I2C peripheral register layout, only the **Register Access Layer** (Section 2’s Layer 4) needs to be rewritten. The Public API, Driver Core, and every line of Application code remain completely unchanged — this is the ultimate payoff of the layered architecture from Section 2.

### 10.8 Scalability Summary Table

| Extension               | Layer(s) That Must Change                                                       | Layer(s) That Stay Identical                                          |
|-------------------------|---------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Polling → Interrupt     | Register Access Layer (new ISR), Driver Core (new <code>_IT</code> entry point) | Public API contract for existing blocking functions, Application code |
| Interrupt → DMA         | Register Access Layer (DMA config)                                              | Public API, Driver Core state machine, Application code               |
| Single → RTOS           | Driver Core (add mutex/semaphore)                                               | Public API signatures, Application code                               |
| Single → Multi-instance | None — already supported                                                        | Everything                                                            |
| STM32L4 → future family | Register Access Layer only                                                      | Public API, Driver Core, Application code                             |

## Section 11 — API Usage Examples

### 11.1 Minimal Application Skeleton

```

#include "stm32l452xx_i2c.h"
#include "stm32l452xx_gpio.h"

static I2C_Handle_t hi2c1;

int main(void)
{
    System_Init();                /* clock tree, SysTick, core setup */

    GPIO_Init();                  /* configure PB8/PB9 for I2C1 AF */

    hi2c1.Instance = I2C1;
    hi2c1.Config.ClockSpeed = 100000; /* 100 kHz standard mode */

    if (I2C_Init(&hi2c1) != I2C_OK)
    {
        /* initialization failed – halt or signal an error indicator */
        while (1) { }
    }

    uint8_t txBuffer[4] = {0x01, 0x02, 0x03, 0x04};

    while (1)
    {
        I2C_Status_t result = I2C_MasterTransmit(&hi2c1, 0x50, txBuffer, 4, 100);

        if (result != I2C_OK)
        {
            /* application-level decision: retry, log, or reset the bus */
            if (I2C_GetError(&hi2c1) == I2C_ERROR_BUS_ERROR)
            {
                I2C_ResetBus(&hi2c1);
            }
        }
    }
}

```

### 11.2 Explaining the Application/Driver Interaction

- `main()` never references `I2C1_BASE`, `CR1`, `CR2`, or any bit mask — it only ever sees `I2C_Handle_t`, `I2C_Status_t`, and named functions. This is Section 1.6’s “application-facing interface” made concrete.
- The application makes a **policy decision** (what to do about a `BUS_ERROR`) while the driver only provides **mechanism** (the ability to detect and recover from that error) — this separation of policy (application) from mechanism (driver) is one of the most important, most transferable ideas in this entire lecture, and reappears verbatim in the Linux kernel and Zephyr comparisons in Section 12.
- The application checks return values and reacts — it never assumes success, directly enacting the defensive-programming and error-reporting pillars from Section 4.1.

## Section 12 — Comparison With Professional SDKs

### 12.1 STM32 HAL

| Aspect          | Our Design                                                                                      | STM32 HAL                                                                                                                                                            |
|-----------------|-------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Handle          | <code>I2C_Handle_t</code> with <code>Instance</code> , <code>Config</code> , <code>State</code> | <code>I2C_HandleTypeDef</code> — structurally almost identical, confirming this is genuinely the industry-standard pattern, not something invented for this course   |
| Naming          | <code>I2C_MasterTransmit</code> , <code>I2C_MasterTransmit_IT</code> (planned)                  | <code>HAL_I2C_Master_Transmit</code> , <code>HAL_I2C_Master_Transmit_IT</code> , <code>HAL_I2C_Master_Transmit_DMA</code> — same suffix convention for mode variants |
| Error reporting | <code>I2C_Status_t</code> return + <code>I2C_GetError()</code>                                  | <code>HAL_StatusTypeDef</code> return + <code>hi2c-&gt;ErrorCode</code> bitfield                                                                                     |
| Callback model  | <code>I2C_RegisterCallback(handle, event, fn)</code>                                            | <code>HAL_I2C_RegisterCallback()</code> (added in later HAL versions) — same idea, added after the fact; we are designing it in from the start                       |
| Scope           | One MCU family, hand-built, minimal                                                             | Entire STM32 portfolio (F0–H7), enormous, generic                                                                                                                    |

**Key architectural takeaway:** our design and STM32Cube HAL converge on nearly identical structural ideas (handle, config struct, callbacks, status/error separation) because these ideas are not arbitrary — they are the *natural consequence* of the constraints (multi-instance support, portability, error handling) that any serious I2C driver must satisfy. HAL is *heavier* because it must serve dozens of chip families and abstraction levels we do not need.

### 12.2 STM32 LL (Low-Layer) Drivers

STM32 LL sits closer to raw registers than HAL — mostly thin, near-1:1 inline functions wrapping individual register bit operations (e.g., `LL_I2C_GenerateStartCondition()`), with **no handle structure and no built-in state machine**. LL is architecturally closer to our **Register Access Layer (Section 2, Layer 4)** than to our complete driver — LL essentially stops at the layer we consider “internal,” and leaves state management, sequencing, and public API design entirely to the application author. This comparison usefully clarifies exactly *what* value our Driver Core layer (Section 2, Layer 3) adds beyond what a vendor already provides for free.

### 12.3 CMSIS

CMSIS provides the register struct definitions (`I2C_TypeDef`) our Register Access Layer depends on, but — as established in Lecture 1 — **CMSIS itself contains no driver logic whatsoever**. CMSIS is pure structure (Layer “below” our Layer 4), never behavior.

### 12.4 Zephyr

Zephyr’s generic `i2c_transfer()` API (introduced in Lecture 1, Section 6.2) takes the layered idea one step further than our design: it inserts a **vendor-agnostic API layer above** the vendor-specific driver, dispatched via a function-pointer table (`struct i2c_driver_api`) resolved at build time from devicetree. Our `I2C_MasterTransmit()` is architecturally equivalent to what sits *underneath* Zephyr’s generic API — Zephyr’s `i2c_stm32.c` driver would, internally, do essentially what our Driver Core and Register Access Layer do. The difference is that Zephyr additionally standardizes the *name* of the public function across all vendors, which our single-vendor course does not need.

### 12.5 Linux I2C Subsystem

Linux formalizes the master-transmit/receive concept as `struct i2c_msg` arrays passed to `i2c_transfer()`, processed by a registered `struct i2c_algorithm` (the STM32-specific equivalent of our entire driver). The **policy/mechanism separation** highlighted in Section 11.2 is Linux’s foundational design philosophy at every layer of the kernel — our small I2C driver is, in effect, a miniature, bare-metal analogue of a Linux bus driver, minus process isolation, virtual filesystem integration, and dynamic driver registration (`probe()` / `remove()`), none of which apply in a single-image bare-metal firmware.

### 12.6 Cross-SDK Comparison Table

| Aspect                        | Our Design              | STM32 HAL              | STM32 LL               | Zephyr                             | Linux I2C                               |
|-------------------------------|-------------------------|------------------------|------------------------|------------------------------------|-----------------------------------------|
| Handle-based                  | Yes                     | Yes                    | No                     | Yes ( <code>struct device</code> ) | Yes ( <code>struct i2c_adapter</code> ) |
| Generic (vendor-agnostic) API | No (STM32L452-specific) | No                     | No                     | Yes                                | Yes                                     |
| Callback support              | Yes (designed in)       | Yes (added later)      | No                     | Yes                                | Yes (completion + async)                |
| State machine                 | Yes                     | Yes                    | No                     | Yes                                | Yes                                     |
| Target scope                  | One project             | Entire STM32 portfolio | Entire STM32 portfolio | Many silicon vendors               | Any Linux-capable hardware              |
| Abstraction cost              | Minimal                 | Moderate–High          | Minimal                | Moderate                           | High (necessarily, for an OS)           |

## Section 13 — Best Practices

### 13.1 Naming Conventions

- Every public symbol carries the module prefix: `I2C_` for functions, types, and enum values (`I2C_Handle_t`, `I2C_Status_t`, `I2C_OK`).
- Verb-first function names describing the action: `I2C_MasterTransmit`, not `I2C_TransmitMaster` or `I2C_TX`.
- Consistent casing: `PascalCase` for types (`I2C_Handle_t`), `PascalCase` for functions (`I2C_MasterTransmit`), `UPPER_SNAKE_CASE` for enum constants and macros (`I2C_ERROR_TIMEOUT`, `I2C_CR1_PE`).

### 13.2 File Organization

Follow Section 3’s structure strictly: one driver, one header/source pair, headers contain only declarations and documentation comments, source files contain all logic. Never declare a function in a `.c` file’s header block that isn’t also declared in the matching `.h`.

### 13.3 Documentation

Every public function should be documented (Doxygen-style comments are the de facto industry standard) with: brief description, `@param` for every parameter, `@return` describing every possible return value, and any preconditions. This documentation *is* the API contract from Section 1.7 made concrete and readable without opening the `.c` file.

### 13.4 Header Hygiene

- Always use include guards (`#ifndef I2C_DRIVER_H` / `#define` / `#endif`), or `#pragma once` consistently per project convention.

- Include only what is directly used in that header (avoid “include everything just in case” — it slows compilation and creates hidden coupling).
- Never define variables (only `extern` declarations) in headers — a variable defined in a header included by multiple `.c` files causes duplicate-symbol link errors.

### 13.5 Avoiding Global Variables

Driver state belongs inside the handle struct, owned by the caller, **not** as a driver-internal global (e.g., avoid `static I2C_Handle_t g_hi2c1`; living inside `i2c_driver.c`). Internal globals silently prevent multi-instance support and make unit testing far harder, since tests cannot reset hidden internal state between runs without a full re-link.

### 13.6 Const Correctness

Parameters that are read but never modified should be marked `const` (e.g., `const uint8_t *pData` for `I2C_MasterTransmit`, since transmit data is only read; contrast with `uint8_t *pData` for `I2C_MasterReceive`, since receive data is written). This documents intent to the reader and lets the compiler catch accidental writes to data that should be read-only.

### 13.7 Static Functions

As established in Section 6, every internal helper is `static`. This is not optional stylistic advice in a professional codebase — treat any non-static internal helper found in code review as a defect to be fixed.

### 13.8 API Versioning

Even without formal semantic versioning, a driver header should carry a version comment or macro (`#define I2C_DRIVER_VERSION "1.0.0"`) so that, as the driver evolves across a multi-year product’s lifetime, engineers can identify exactly which contract a given piece of application code was written against — critical during long-term maintenance and when merging branches that may have diverged driver versions.

### 13.9 Code Readability

Favor clarity over cleverness: a slightly longer, obviously-correct polling loop is preferable to a compact but opaque one. Embedded code is read far more often (by code reviewers, by yourself six months later, by whoever maintains the product after you) than it is written.

### 13.10 Maintainability

Achieved by the cumulative effect of everything in this lecture: layering (Section 2), clean folder structure (Section 3), a small stable public API (Section 5), hidden internals (Section 6), and explicit configuration/state/error handling (Sections 7 and 9). Maintainability is not a separate technique — it is the *outcome* of correctly applying all the others.

### 13.11 MISRA-C Considerations

MISRA-C is a widely used coding standard in safety-critical and automotive embedded C, and several of its rules directly reinforce this lecture’s architecture:

| MISRA-C theme                                                           | How our design already complies                                                                          |
|-------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| No implicit type conversions                                            | Every parameter is explicitly and appropriately typed ( <code>uint16_t</code> , <code>uint8_t *</code> ) |
| Avoid dynamic memory allocation                                         | Handles are caller-allocated (static/global), never <code>malloc</code> ’d inside the driver             |
| Every function has a single, well-defined exit strategy where practical | Our flowcharts (Section 8) show clear, bounded decision paths with explicit returns                      |
| No unreachable/dead code                                                | The error-enum design (Section 9.9) avoids vague catch-all branches                                      |
| Restrict use of global variables                                        | Section 13.5                                                                                             |

### 13.12 Defensive Programming

Validate first, act second, always — covered in depth in Sections 4.1 and 9.5, worth restating here as a cross-cutting practice that touches every function in the driver, not just the “important” ones.

## Section 14 — Common Beginner Mistakes

| # | Mistake                                                                                                                                           | Why It's Problematic                                                                                                                               | How to Avoid It                                                                                                                                                                                    |
|---|---------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <b>Exposing registers directly</b> — putting <code>I2C1-&gt;CR1</code> access inside <code>main.c</code> or application-level files               | Destroys ownership, portability, and testability (Lecture 1, Section 3.6); makes future porting to another chip require rewriting application code | Route every hardware interaction through the public API; treat direct register access outside the Register Access Layer as an automatic code-review rejection                                      |
| 2 | <b>Poor API names</b> — <code>i2c_go()</code> , <code>i2c_do_stuff()</code> , <code>i2c_x()</code>                                                | Forces every caller to read the implementation to understand behavior, defeating the entire purpose of an API (Section 1.7)                        | Verb+Noun naming, module prefix, consistent casing (Section 13.1)                                                                                                                                  |
| 3 | <b>Tight coupling</b> — the I2C driver directly manipulating GPIO registers instead of calling the GPIO driver's API                              | Any future GPIO driver change (e.g., adding pull-up config) risks silently breaking the I2C driver too                                             | Depend only on other drivers' public APIs, never their internals (Lecture 1, Section 7.5)                                                                                                          |
| 4 | <b>Mixing application and driver code</b> — sensor-specific logic (e.g., temperature conversion formulas) living inside <code>i2c_driver.c</code> | Destroys reusability — the driver becomes useless for any other I2C device on a future project                                                     | Keep the I2C driver protocol-generic; device-specific logic belongs in a separate device driver layer (e.g., a <code>bmp280.c</code> built <i>on top of</i> the I2C driver)                        |
| 5 | <b>Global variable misuse</b> — a single hidden global handle inside the driver source file                                                       | Prevents multi-instance support; makes concurrent access bugs far more likely and far harder to trace                                              | Handles are caller-owned and passed explicitly to every call (Section 13.5)                                                                                                                        |
| 6 | <b>Ignoring error handling</b> — calling driver functions without checking return values                                                          | Silent failures propagate until they cause a much more confusing downstream symptom                                                                | Treat every non- <code>void</code> driver function's return value as mandatory to check, enforced by code review or static analysis ( <code>-Wunused-result</code> -style tooling where available) |
| 7 | <b>Hardcoding addresses</b> — writing <code>*(volatile uint32_t*)0x40005400 = ...</code> instead of using CMSIS structs                           | Unreadable, untyped, and completely defeats the compiler-checking benefit CMSIS exists to provide (Lecture 1, Section 5.8)                         | Always go through CMSIS-provided struct/macro names                                                                                                                                                |
| 8 | <b>No timeout mechanisms</b> — <code>while (!(I2C1-&gt;ISR &amp; FLAG));</code> with no bound                                                     | A single misbehaving or absent slave device permanently hangs the entire system                                                                    | Every wait loop must be bounded and return a timeout error (Section 9.2)                                                                                                                           |
| 9 | <b>No abstraction</b> — one giant <code>main()</code> containing both business logic and bit-level protocol sequencing                            | Impossible to reuse, impossible to unit test, impossible for a second engineer to safely modify without re-understanding everything at once        | Apply the full layered architecture from Section 2, even for “simple” projects — the discipline pays off the moment the project grows even slightly                                                |

## Section 15 — Interview Questions

### 15.1 Basic Questions (1-20)

- What is an API, in one sentence?** A documented contract describing what a module promises to do, without exposing how it does it internally.
- Why should application code never call driver internal (static) functions?** Because internal functions perform partial, unsequenced steps with no state validation; bypassing the public API can leave hardware in an inconsistent or undefined state.
- What is the difference between a public API and a private API?** A public API is declared in the header file and intended for external use; a private API is `static`, confined to the source file, and invisible to the linker outside that file.
- Why does the I2C driver use a handle structure?** So the same driver code can control multiple independent I2C instances (I2C1/I2C2/I2C3), with instance identity and runtime state carried in the handle argument rather than hardcoded.
- What does a configuration structure represent?** The set of user-selectable parameters (clock speed, addressing mode, etc.)

needed before initialization, bundled into one struct instead of a long parameter list.

**6. Why is `I2C_Init()` separate from `I2C_Enable()`?** `Init` performs full one-time setup (clocks, GPIO, timing calculation); `Enable` only toggles the peripheral enable bit, useful for pausing/resuming without full reconfiguration.

**7. What problem does a callback solve?** It lets the driver notify the application of an event (completion, error) without needing to know what the application wants to do in response — inversion of control.

**8. Why must every wait loop have a timeout?** Without one, a misbehaving or absent slave device can hang the system permanently; a timeout guarantees the function eventually returns with a reportable error.

**9. What is the difference between `I2C_MasterTransmit` and `I2C_MemWrite`?** `MasterTransmit` sends raw bytes to a device address; `MemWrite` additionally frames an internal memory/register address (with a repeated START where needed) before sending data — a common pattern for EEPROMs and sensor register maps.

**10. Why is `I2C_ScanBus()` a development-time tool rather than a production one?** It probes every possible address on the bus, which is useful for discovering connected hardware during bring-up but wastes time/flash and can confuse address-sensitive devices in production.

**11. What is the purpose of `I2C_GetState()`?** It gives read-only, encapsulated visibility into the driver’s current state machine condition, without exposing the handle’s internals directly to modification.

**12. Why should `pData` be `const` in `I2C_MasterTransmit` but not in `I2C_MasterReceive`?** `Transmit` only reads the buffer (const-correct); `receive` writes into the buffer, so it cannot be `const`.

**13. What is the driver folder structure’s `Inc / Src` split for?** It mirrors the public/private (contract/implementation) distinction at the filesystem level, making the project’s architecture visible just from browsing directories.

**14. Why shouldn’t the I2C driver directly configure GPIO registers?** That would tightly couple the I2C driver to GPIO internals; the I2C driver should call the GPIO driver’s public API instead, preserving modularity and loose coupling.

**15. What is the difference between a return code and a status?** A return code answers “did this one call succeed?”; a status answers “what is the peripheral doing right now?” — queryable independently of any single call.

**16. Why does the driver need both a return code and a separate error code accessor (`I2C_GetError`)?** The return code communicates immediate outcome; the error code persists the specific failure reason for later diagnostic inspection, since a caller may check state after the fact.

**17. What happens if a public API function is called before `I2C_Init()`?** It should detect the un-initialized state (`I2C_STATE_RESET`) and return `I2C_ERROR_NOT_INITIALIZED` rather than touching hardware.

**18. Why is `I2C_DeInit()` needed if the MCU can simply be reset?** Full MCU reset is not always appropriate (e.g., reconfiguring one peripheral without disturbing the rest of a running system, or preparing for low-power modes) — `DeInit` allows clean, selective teardown.

**19. What is the purpose of a default configuration (Section 7.7)?** It lowers the barrier to correct first use by giving a known-good starting configuration that the caller can selectively override.

**20. Why must internal helper functions be `static`?** `static` gives internal linkage, so the symbol is invisible to the linker outside its translation unit — enforced by the compiler/linker, not just convention, preventing external code from calling unsequenced internal steps.

## 15.2 Intermediate Questions (21-40)

**21. Explain why compile-time and run-time configuration are both needed, with an example of each.** Compile-time configuration (e.g., `#define I2C_MAX_INSTANCES`) fixes resource limits and feature toggles that never change on a given target and benefit from smaller/faster code; run-time configuration (e.g., `ClockSpeed` in the config struct) covers decisions the application must make dynamically, such as which sensor is attached.

**22. Why does the driver’s error enum avoid a generic `I2C_ERROR_UNKNOWN` value?** A catch-all error code hides the actual cause, forcing every caller into the same limited response regardless of what really happened; each distinct failure mode should map to a distinct, actionable error code.

**23. What is the architectural difference between a retry mechanism and a bus-recovery mechanism?** A retry mechanism re-attempts a recoverable protocol-level failure (a single NACK/timeout) a bounded number of times at the Driver Core layer; bus recovery is a last-resort GPIO-level procedure that forcibly clears a stuck bus condition when simple retries have failed.

**24. Why should assertions be distinguished from return-code error handling?** Assertions catch impossible-if-correct internal logic errors (programmer bugs) typically only in debug builds, while return codes handle expected runtime failures (bad input, hardware timeout) that must always be handled, in every build configuration.

**25. Explain the difference between the Driver Core layer and the Register Access Layer.** The Driver Core validates parameters/state and sequences a series of steps to fulfil an API contract; the Register Access Layer performs the individual mechanical register operations that Driver Core sequences — only this layer touches hardware registers directly.

**26. Why is `I2C_RegisterCallback()` designed now, before interrupt-mode is implemented?** So the blocking (polling) implementation built first does not require an incompatible API redesign once interrupt-driven and DMA-driven modes are added later — designing the mechanism early preserves backward compatibility for future extensions.

**27. What does “policy vs mechanism” mean, and where does it appear in this driver’s design?** Mechanism is the capability the driver provides (e.g., detecting and recovering from a bus error); policy is the decision about what to do with that capability, made by the



application (e.g., deciding whether to retry, log, or alert). The driver should provide mechanism, the application should decide policy.

**28. Why does `I2C_MemRead()` require a repeated START instead of a STOP followed by a new START?** Many I2C devices require the bus master to retain control of the bus between selecting an internal memory address and reading from it; a STOP would release the bus and could allow another device to interject, or cause the target device to reset its internal address pointer.

**29. Explain why the same handle-based architecture supports multi-instance operation “for free.”** Because no function above the Register Access Layer ever hardcodes a specific peripheral instance — all instance-specific state (the `Instance` pointer, `State`, buffers) lives in the caller-supplied handle, so creating additional handles for I2C2/I2C3 requires no new driver code.

**30. What would need to change to add DMA support, and what would not?** Only the Register Access Layer changes (configuring a DMA channel instead of polling/writing bytes directly); the public API signatures, state machine, and callback mechanism remain unchanged, with a new `_DMA`-suffixed entry point added additively.

**31. Why is a config struct preferred over a long parameter list for `I2C_Init()`?** It scales gracefully as new configuration options are added (via designated initializers) without breaking existing call sites, and avoids the error-prone risk of misordering many individual arguments.

**32. What is the risk of a callback that calls back into the same handle’s transfer function from within the callback itself?** It risks re-entrancy — calling a transfer function while the driver’s state machine has not yet returned to a stable state, potentially corrupting state or violating the state machine’s assumptions.

**33. Why should the `I2C_HandleError()` function be centralized rather than inlined at every failure point?** Centralizing error handling ensures consistent behavior (state update, error recording, optional recovery trigger) everywhere a failure can occur, instead of risking inconsistent, duplicated logic scattered throughout the driver.

**34. What does “information hiding” (Parnas) mean, applied to the TIMINGR calculation?** Design decisions likely to change (the exact algorithm mapping a requested clock speed to register bits) should be hidden inside the driver, so that if the algorithm or the target family changes, only the driver’s internals need to change — not every application that calls `I2C_Init()`.

**35. Why is `I2C_IsDeviceReady()` distinct from `I2C_ScanBus()`?** `ScanBus` is a development-time discovery tool probing all addresses; `IsDeviceReady` is a production-relevant, targeted poll of one known device address, useful for waiting out a device’s internal busy period (e.g., an EEPROM write cycle).

**36. What is the build-process reason `static` functions cannot be called from other files?** Each `.c` file compiles independently into an object file; `static` gives internal linkage, so the symbol is excluded from that object file’s exported symbol table, and the linker has no way to resolve a reference to it from another object file.

**37. Why does the driver validate parameters before checking peripheral state, or does order not matter?** Order can vary by convention, but a consistent rule should be documented and applied everywhere (e.g., validate parameters first) so that error precedence is predictable and callers can rely on receiving `INVALID_PARAM` rather than a state-related error when both conditions are true.

**38. Explain why MISRA-C’s discouragement of dynamic memory allocation aligns with this driver’s handle design.** Handles are caller-allocated (typically statically or globally), never `malloc`’d inside the driver, which avoids heap fragmentation and non-deterministic allocation failure — both significant concerns in resource-constrained and safety-critical embedded systems.

**39. What is the purpose of documenting a driver’s version (Section 13.8)?** It lets engineers identify exactly which API contract a piece of application code was written against, which matters across a multi-year product lifetime and when reconciling divergent branches.

**40. Why is a “thin” `main.c` (Section 3.1/11.1) considered good practice?** Keeping `main.c` thin (delegating real logic to `Application/`) preserves the separation between the entry point and business logic, so the entry point stays stable even as application logic evolves substantially.

## 15.3 Advanced Design Questions (41-60)

**41. Design justification: why does this course’s layered model place the Register Access Layer below the Driver Core rather than merging them?** Merging them would force every state/parameter-validation code path to also contain direct register manipulation, defeating the “one auditable place for hardware-specific logic” benefit and making it much harder to reason about or test the state machine independently of real hardware.

**42. If you needed to support both a “safe blocking” and a “fast unsafe” transmit function for performance-critical code, how would you extend this API without violating its design philosophy?** Add an additively-named function (e.g., `I2C_MasterTransmitFast()`) with clearly documented relaxed guarantees (e.g., no parameter validation), rather than modifying the existing `I2C_MasterTransmit()` contract — preserving backward compatibility (Section 4.1) while still meeting the new requirement.

**43. How would you unit-test the Driver Core’s state machine logic without real STM32 hardware?** By defining a mock/stub Register Access Layer (replacing real register writes with recorded calls or simulated return values) linked against the same Driver Core source, allowing the state transitions and validation logic to be exercised and asserted against on a PC build.

**44. Explain how you would evolve this driver’s error enum over time without breaking existing application code.** Only append new values to the end of the enum (never renumber or remove existing ones), and treat the enum, once shipped, as append-only — exactly analogous to backward compatibility rules for any versioned public interface.

**45. Why might a safety-critical variant of this driver add a CRC or software watchdog check around bus recovery, and where would that logic belong architecturally?** To detect and bound worst-case recovery time/failure count in safety-relevant applications; this logic belongs in the Driver Core (orchestration and policy around recovery), not the Register Access Layer (which only performs

individual mechanical steps).

**46. How would you architect this driver to support both 7-bit and 10-bit addressing without an API break?** Represent addressing mode as a configuration field ( `AddressingMode` in `I2C_Config_t` ) rather than encoding it in the function name or signature, so existing calls to `I2C_MasterTransmit()` continue to work unchanged regardless of addressing mode chosen at init time.

**47. What is the architectural argument for keeping `I2C_ScanBus()` compiled out of release builds rather than just leaving it unused?** Leaving it compiled in still consumes flash space and remains callable, meaning a stray or malicious call path in production code could still trigger unwanted bus traffic; a build-time exclusion ( `#ifdef I2C_DEBUG` ) removes the capability entirely, not just discourages its use.

**48. Explain, architecturally, why a driver author should resist adding “just one more parameter” directly to `I2C_MasterTransmit()`’s signature for a rarely-used feature.** Each additional required parameter increases the burden on every existing call site and increases the chance of misuse; a rarely-used feature is better served by an optional configuration field, a separate variant function, or a callback/registration mechanism, preserving the core function’s simplicity (Section 4.1).

**49. How does the “single responsibility per internal function” principle (Section 6.2) aid debugging in the field?** When a bug is reported (e.g., “STOP condition never generated”), a developer can go directly to the one function responsible ( `I2C_GenerateStop` ) rather than searching through an undifferentiated block of sequential register writes.

**50. Why is it architecturally significant that the Application layer never directly depends on Core/ (Section 3.3)?** It confirms the whole-system clock/startup configuration is decoupled from business logic, meaning the application could in principle run correctly-configured against a different startup/clock setup without modification, as long as the driver contracts are honored.

**51. Explain the trade-off of designing `I2C_RegisterCallback()` around an event enum parameter versus separate registration functions per event type.** An event enum parameter ( `I2C_RegisterCallback(handle, I2C_EVENT_TX_COMPLETE, fn)` ) keeps the public API surface smaller and more uniform as new event types are added; separate functions per event ( `I2C_RegisterTxCompleteCallback()` , etc.) are more discoverable/self-documenting but multiply the API surface with every new event type — a real trade-off between uniformity and discoverability.

**52. How would you justify, to a skeptical teammate, spending a full lecture on architecture before writing any register code?** Point to the scalability analysis in Section 10: every future extension (interrupt mode, DMA, multi-instance, new silicon family) is additive rather than a rewrite specifically because the architecture was designed for those extensions up front — the time invested here is repaid many times over across the project’s lifetime.

**53. What does it mean for a driver’s public API to be “append-only” over its lifetime, and why does that matter for embedded products with long support windows?** It means new capabilities are added as new functions/fields/enum values, and existing ones are never removed or semantically changed, so firmware shipped years ago (and no longer under active development) continues to compile and behave identically against newer driver versions used in newer product variants.

**54. Why does concentrating all register access in one layer (Section 2, Layer 4) matter specifically for handling silicon errata?** Because a chip errata workaround (a documented silicon bug requiring a specific sequence or extra delay) needs to be applied in exactly one place; if register access were scattered across the codebase, the same errata fix would need to be found and applied in every scattered location, with high risk of missing one.

**55. Design question: how would you extend the handle structure to support a “last activity timestamp” for a watchdog feature, without breaking existing code?** Add a new field (e.g., `uint32_t LastActivityTick;` ) to the end of `I2C_Handle_t` — existing code that only reads/writes previously-existing fields is unaffected, since C struct field addition at the end does not change the meaning of existing field accesses.

**56. Why should the Driver Core, not the application, own retry-count and backoff logic for transient I2C errors?** Retry logic requires intimate knowledge of which errors are transient/protocol-level versus which are fatal/bus-level — knowledge that belongs inside the driver, which owns the state machine and error classification; exposing this decision to every application would duplicate the same fragile logic across every user of the driver.

**57. What is the risk of allowing `I2C_GetState()` / `I2C_GetError()` to return direct pointers into the handle rather than copied values?** Returning direct pointers/references risks the caller mutating internal driver state through what should be a read-only accessor, breaking encapsulation; returning copied values (as designed) preserves the driver’s exclusive control over its own internal state.

**58. Explain why the folder structure’s dependency rule (“Drivers/ must never include Application/”) is enforceable, not just a guideline.** It can be enforced mechanically via build-system include-path restrictions (only exposing `Drivers/Inc` and `Core/Inc` to the compiler when building `Drivers/Src` , never `Application/Inc` ), turning an architectural rule into a build error if violated, rather than relying purely on code-review discipline.

**59. How would the presence of an RTOS change the answer to “who owns retry/backoff timing” from Question 56?** Under an RTOS, timing/backoff could be implemented as a task delay ( `vTaskDelay` ) rather than a busy-wait, and the retry logic might interact with a semaphore given by the ISR — but the ownership of the decision (Driver Core, not application) remains the same; only the underlying timing mechanism changes (Section 10.5).

**60. Why is designing the public API before the internal implementation considered a hallmark of senior engineering practice, rather than merely a stylistic preference?** Designing the contract first forces explicit reasoning about every caller-visible behavior, error case, and long-term compatibility concern before any implementation detail can bias or constrain those decisions; it is the practical application of “program to an interface, not an implementation,” a principle with decades of demonstrated payoff across the software industry, not unique to embedded systems.

## 15.4 Architecture Scenario Questions (61–70)



- 61. Scenario: A teammate proposes adding a `void *userData` field to `I2C_Handle_t` so their application can stash a pointer for use inside callbacks. Evaluate this design.** This is a well-established, reasonable pattern (seen in STM32Cube HAL’s callback context, Zephyr’s device data pointers, and Linux’s `private_data` conventions) — it allows generic callback signatures to still reach application-specific context without the driver needing to know what that context is; the only caution is ensuring the field is clearly documented as application-owned and never touched by the driver itself.
- 62. Scenario: Your driver’s `I2C_MasterTransmit()` is called from both the main loop and, indirectly, from an interrupt context in a mixed polling/interrupt codebase during a transition period. What problem would you expect, and how would you address it architecturally?** Expect state-machine corruption from concurrent access to the same handle from two execution contexts; address it by guarding state transitions (a simple busy-flag check is not sufficient against true concurrency) and, if interrupt-context calls are genuinely required, redesigning that specific interaction to use the interrupt-mode API and callback mechanism (Section 10.3) rather than calling the blocking function from an ISR at all (which is itself close to always the wrong answer for a blocking API).
- 63. Scenario: A future requirement demands supporting a second, completely different I2C-capable device (e.g., an external I2C-to-USB bridge chip) alongside the STM32’s built-in peripheral, under the same application code. How does the layered architecture accommodate this?** This is exactly the scenario Zephyr’s generic-API-plus-function-pointer-table model (Section 12.4) solves elegantly; our simpler design could be extended by introducing a thin abstract interface above the current public API (a struct of function pointers implemented by both the STM32-native driver and a new bridge-chip driver), without needing to change either underlying implementation.
- 64. Scenario: A safety auditor asks you to demonstrate that no code outside the I2C driver can corrupt I2C1’s hardware state. How does this architecture support that claim?** Because the Register Access Layer is the only code permitted to write I2C registers (enforced by code organization and, ideally, static analysis tooling checking for register access outside `Drivers/Src/stm32l452xx_i2c.c`), an auditor can restrict their register-level review to that one file, while reviewing all other layers purely as ordinary state-machine/application logic.
- 65. Scenario: Product management wants the driver to support a “silent mode” that never blocks, always returning immediately with a queued-status result. How would you fit this into the existing API design without a breaking change?** Add a new configuration option or a new function variant (e.g., a non-blocking `I2C_MasterTransmit_NonBlocking()` returning `I2C_PENDING`), following the same additive-extension pattern already used for interrupt/DMA modes (Section 10), rather than changing the blocking behavior of the existing function.
- 66. Scenario: During integration, another engineer’s middleware module begins calling `I2C_GenerateStart()` directly because “it was faster than going through the full API.” What is the architectural failure here, and how do you prevent recurrence?** The failure is that an internal helper was made accidentally public (not properly `static`) or was exposed in the header by mistake — prevent recurrence by strictly enforcing the public/private split (Section 6), reviewing header file contents specifically for accidental over-exposure, and treating any direct call to an internal function from outside the driver’s own source file as a code-review-blocking defect.
- 67. Scenario: A new STM32 family is announced with a redesigned I2C peripheral that supports only DMA-based transfers, not simple register polling. How much of this architecture survives the port?** The Public API, Driver Core state machine, error/status model, and callback mechanism all survive unchanged; only the Register Access Layer is rewritten to always route through DMA internally — this is the direct, concrete payoff of Section 10.7’s scalability claim.
- 68. Scenario: You discover the current design’s `I2C_MemRead()` cannot support a memory address size larger than 16 bits, but a new device requires a 24-bit internal address. How do you evolve the API?** Widen the `memAddress` parameter type (or, more robustly, redesign it as a small byte-array parameter with an explicit length) in a new, additively-named function or an extended function signature guarded by careful backward-compatibility analysis — this scenario also illustrates why choosing sufficiently generic parameter types up front (Section 4.1’s “extensibility” pillar) reduces the likelihood of needing a breaking change later.
- 69. Scenario: A regulatory requirement now mandates a persistent log of every I2C error, including timestamp and error type, retained across resets in non-volatile memory. Where does this logic belong?** This is a middleware-layer concern (Lecture 1, Section 4.2), not a Driver Core or Register Access Layer concern — the I2C driver should simply provide a suitably detailed error via its existing callback/error-reporting mechanism, and a separate logging middleware module (built on top of the I2C driver’s public API, with its own dependency on a non-volatile storage driver) should own persistence and formatting.
- 70. Scenario: Leadership asks whether this bare-metal driver architecture could realistically be “graduated” into a Zephyr-style generic driver model if the product line adds a second silicon vendor. Assess the feasibility based on this lecture’s design.** Highly feasible: because the public API is already clean, handle-based, and free of STM32-specific leakage into the Application layer (Sections 2 and 5), the natural evolution is to introduce a vendor-agnostic function-pointer-table layer above the existing public API (mirroring Section 12.4’s Zephyr comparison), with the current implementation becoming the STM32-specific “backend” behind that new generic interface — requiring no rewrite of Application-layer code at all.

---

## Section 16 — Exercises

---

### 16.1 Theory Exercises

1. In your own words, explain the difference between a return code, a status, and an error code, using a scenario where all three are queried after a single failed transfer.
2. Explain why information hiding (Parnas) is a stronger idea than simple encapsulation, using the `TIMINGR` calculation as your example.
3. Describe, without notes, the five layers of this lecture’s architecture (Section 2) and state one responsibility for each.

4. Explain why the Register Access Layer is described as “the only place errata fixes need to be applied,” using a hypothetical silicon bug as your example.

## 16.2 API Design Exercises

5. Design the full function signature and documentation-style contract (purpose, parameters, return type, preconditions) for a new function `I2C_SlaveTransmit()`, assuming the STM32L452’s I2C peripheral is configured to act as a slave instead of a master.
6. Propose a configuration struct field set for supporting both Standard Mode (100 kHz) and Fast Mode Plus (1 MHz) operation, and justify each field.
7. Design (signature only) a function that allows an application to query how many bytes remain in an in-progress interrupt-mode transfer, and explain which layer this data must be tracked in.
8. Critique this signature: `int I2C_Send(int id, char *d, int n);` — list every design flaw relative to the principles in Section 4, and rewrite it correctly.

## 16.3 Code Review Exercises

9. You are reviewing a pull request that adds this line inside `app.c`: `I2C1->CR2 |= I2C_CR2_START;`. Write the code review comment you would leave, referencing specific sections of this lecture.
10. A pull request adds a new public function `I2C_Debug_DumpRegisters()` with no `#ifdef` guard, intended for development use only. Identify the architectural issue and propose the correct fix.
11. A pull request changes `I2C_Status_t I2C_MasterTransmit(...)` to `void I2C_MasterTransmit(...)`, arguing “it always succeeds in our testing.” Write your review response.

## 16.4 Driver Architecture Exercises

12. Draw (ASCII, as in Section 8) a complete flowchart for `I2C_IsDeviceReady()`, including the retry-count loop and timeout handling.
13. Extend the folder structure from Section 3 to include a new `bmp280_driver` device driver built on top of the I2C driver, and show the correct include hierarchy and dependency-flow diagram for it.
14. Design the state machine (ASCII diagram, as in Lecture 1 Section 7.11) for `I2C_MemRead()`, including the repeated-START transition explicitly as its own state.

## 16.5 Refactoring Exercises

15. Refactor this (intentionally bad) function into a properly layered public function plus internal helpers, following Section 2 and Section 6:

```
void I2C_Send(uint8_t addr, uint8_t *data, uint8_t len)
{
    I2C1->CR2 |= (1 << 8);
    while (!(I2C1->ISR & (1 << 1)));
    I2C1->TXDR = addr << 1;
    for (int i = 0; i < len; i++)
    {
        while (!(I2C1->ISR & (1 << 0)));
        I2C1->TXDR = data[i];
    }
    I2C1->CR2 |= (1 << 14);
}
```

16. Identify every principle from Sections 4, 6, and 9 that the function above violates, and produce a checklist a code reviewer could reuse for future submissions.

---

## Section 17 — Mini Assignment

### Assignment: Design the API Architecture for a Reusable SPI Driver

Using **exactly the same architectural principles** developed in this lecture — layering (Section 2), folder structure (Section 3), API design philosophy (Section 4), public API design with full per-function justification (Section 5), internal function design (Section 6), configuration philosophy (Section 7), execution-flow flowcharts (Section 8), error handling strategy (Section 9), and scalability planning (Section 10) — design (architecture only, no implementation) a reusable SPI driver for the STM32L452RE.

#### Your submission should include:

1. A layered architecture diagram (like Section 2.1) specific to SPI, correctly identifying what “master transmit” and “master receive” mean for SPI’s full-duplex nature (unlike I2C’s half-duplex, addressed communication).
2. A proposed folder structure (like Section 3.1) fitting SPI into the existing `Drivers/` organization.
3. A complete public API function list (like Section 5), with full justification for each function, explicitly considering: does SPI need an equivalent of `I2C_ScanBus()` or `I2C_IsDeviceReady()`? Why or why not?
4. A configuration structure design (like Section 7), correctly identifying SPI-specific parameters (clock polarity, clock phase, bit order,

data size, NSS management) that have no I2C equivalent.

5. At least three execution-flow flowcharts (initialization, full-duplex transfer, shutdown).
6. An error-handling strategy (like Section 9), specifically addressing what “bus recovery” even means for SPI (a bus without inherent addressing or arbitration, unlike I2C) — is a recovery mechanism even necessary, and why or why not?
7. A scalability discussion (like Section 10) covering polling → interrupt → DMA → multi-instance (SPI1/SPI2/SPI3) → RTOS.

**Do not implement any register-level code.** The deliverable is architecture and API design, exactly matching the scope and rigor of this lecture.

---

## Summary

### What We Covered

- An API is a contract, not a description of implementation — it exists to freeze a stable boundary that allows independent development, safe internal change, and testability.
- A five-layer architecture (Application → Public API → Driver Core → Register Access Layer → Hardware) cleanly separates *what* is requested from *how* it is physically achieved, with dependencies flowing strictly downward.
- A professional folder structure ( `Drivers/Inc` , `Drivers/Src` , `Core/` , `Application/` ) enforces the public/private and layering distinctions at the filesystem level, not just in code.
- Every public API function was designed with a full contract — purpose, parameters, return type, internal operation, common mistakes, and example usage — *before* any implementation exists.
- Internal helper functions are deliberately `static` , concentrating all register access in one auditable Register Access Layer.
- Configuration objects, handles, state variables, callbacks, and configurable timeouts each solve a specific, distinct engineering problem — none of them are decorative.
- Complete execution-flow flowcharts were designed for initialization, master transmit/receive, memory read/write, error recovery, and shutdown — all traceable from `main()` down to conceptual register access.
- A layered, three-tier error-handling model (return codes, status, error codes) plus timeouts, retries, bus recovery, and assertions together form a robust, defensive error strategy.
- The architecture was explicitly proven scalable — polling, interrupt, DMA, RTOS integration, multi-instance support, and future silicon families all require only additive or Register-Access-Layer-only changes, never a public API rewrite.
- Comparing this design against STM32 HAL, STM32 LL, CMSIS, Zephyr, and the Linux I2C subsystem confirmed that our architectural choices are not arbitrary — they converge with real, battle-tested industry SDKs because they solve the same underlying engineering constraints.

### Key Takeaways

***An API is designed before it is implemented — every function in this lecture has a complete, documented contract, and not a single register has been touched yet.***

***Every architectural element (handle, config struct, state machine, callback, layered folder structure) exists to solve one specific, nameable engineering problem. If you cannot name the problem an architectural element solves, it does not belong in the design.***

***A well-layered driver is scalable by construction: interrupt mode, DMA mode, RTOS integration, multi-instance support, and future silicon ports should all be additive changes, confined mostly or entirely to the Register Access Layer — never a rewrite of the public API or the application code built on top of it.***

### What Comes Next

Lecture 5 will take this fully justified architecture and begin **real implementation** against the STM32L452’s actual I2C registers ( `CR1` , `CR2` , `OAR1` , `OAR2` , `TIMINGR` , `ISR` , `TXDR` , `RXDR` ) as defined in RM0394 — starting with the `I2C_Config_t` and `I2C_Handle_t` struct definitions designed conceptually in this lecture, and the real register-level logic behind `I2C_Init()` .

---

*End of Lecture 4 — Designing a Professional I2C Driver Architecture*

*Stopping here as instructed. Awaiting the next prompt before continuing to Lecture 5.*